Nonlinear Tucker Decomposition with Polynomial Links

Kaiyu Shen

Abstract

Classical Tucker decomposition takes multilinear low-rank structure as its core. However, when an observed tensor is generated by nonlinear mechanisms, saturated responses, complex higher-order interactions, or heteroscedastic perturbations, the purely multilinear assumption often leads to systematic mismatch. To address this issue, this paper proposes Nonlinear Tucker Decomposition with Polynomial Links (NTD-PL). The model retains a shared Tucker latent tensor as the low-rank backbone and imposes an explicit polynomial link on top of it, thereby incorporating standard Tucker into a unified framework as a linear special case and enabling a controlled extension toward higher-order interactions through only a small number of link coefficients.

On the theoretical side, we clarify the nesting relation between NTD-PL and Tucker, establish exact representation results under polynomial generation, and derive finite-order truncation approximation bounds under analytic generation. We also formulate a unified masked learning objective and, together with Tucker initialization, gradient updates, and coefficient updates for the polynomial link, obtain a practical alternating optimization method. Experimental results show that, on linear source data, NTD-PL almost coincides with Tucker and does not create artificial gains; under a unified nonlinear residual protocol, NTD-PL achieves stable improvements in both structure-matching and finite-order approximation settings; and on real hyperspectral reconstruction and random-missing completion tasks, NTD-PL consistently outperforms Tucker under the same parameter budget. Further mechanism studies show that these gains can be attributed neither to simply adding parameters nor to applying a single shared polynomial correction to the Tucker output.

Overall, this paper provides an explicit, low-dimensional, and analyzable structured solution for enhancing nonlinear generation mechanisms while preserving the Tucker backbone.

1 Introduction

Tensor decomposition provides a natural low-rank representation for multiway data and has been widely used in image restoration, signal processing, compact representation, and missing-data completion tasks [2, 5, 13, 17, 25]. As one of the classical models, Tucker decomposition characterizes couplings across modes through a core tensor and mode-wise factor matrices, achieving a favorable balance among structural interpretability, flexibility of rank control, and algorithmic feasibility [6, 13, 26]. However, standard Tucker decomposition is essentially built on a multilinear entry-generation assumption. When the observed tensor is produced by nonlinear generation, saturated responses, complicated higher-order interactions, or heteroscedastic perturbations, a purely multilinear model often leads to systematic mismatch, which in turn limits its performance in reconstruction, recovery, and downstream prediction tasks.

To alleviate this problem, a variety of nonlinear tensor decomposition methods have appeared in recent years. Existing work modifies the mapping from latent linear predictors to observed entries through link functions, generalized likelihoods, kernel methods, or Gaussian processes on the one hand, and enhances model expressivity through deep priors, neural decoders, or networkized decomposition structures on the other hand [11, 22, 27, 31, 34]. These methods show that nonlinear extensions can indeed substantially improve the ability of tensor models to adapt to complex data-generating mechanisms. At the same time, however, they are often accompanied by higher inference and training complexity, a larger hyperparameter design space, or a weakening of the structural transparency of classical Tucker decomposition. It therefore remains meaningful to ask whether one can introduce controlled yet effective nonlinear expressive power into Tucker while preserving, as much as possible, its low-rank backbone, parameter efficiency, and multiway interpretability. This is precisely the starting point of this paper.

We propose Nonlinear Tucker Decomposition with Polynomial Links (NTD-PL). The model uses a shared Tucker latent tensor

$$\mathcal{S} = \mathcal{X} \times_1 \mathbf{A}^{(1)} \times_2 \cdots \times_N \mathbf{A}^{(N)},$$

as the low-rank backbone, and imposes on it an explicit element-wise polynomial link

$$\widehat{\mathcal{Y}} = f_{\boldsymbol{\beta}}(\mathcal{S}) = \sum_{p=0}^{P} \beta_p \mathcal{S}^{\odot p}.$$

In this way, standard Tucker is naturally included as a linear special case, while higher-order nonlinearity is introduced in a controlled manner through a small number of link coefficients. Unlike nonparametric mappings based on kernels or Gaussian processes, the nonlinearity in NTD-PL is explicit, low-dimensional, and deterministic. Unlike deep nonlinear tensor models that rely on neural decoders, NTD-PL does not replace the low-rank Tucker backbone; instead, it only strengthens the entry-generation mechanism in a structured manner. It is therefore better understood as a model that enhances Tucker rather than completely replaces it.

The key point of this construction is that it is not simply an element-wise post-processing step applied to a Tucker reconstruction. Since each entry of the latent tensor $\mathcal{S}$ is already generated by multilinear coupling between the core tensor and the mode factors, $\mathcal{S}^{\odot p}$ corresponds to a recombination of higher-order interactions on the shared low-rank backbone rather than to an external correction independent of the low-rank structure. Precisely because of this, NTD-PL can achieve expressive power beyond purely multilinear models without explicitly introducing higher-order factor tensors, while still preserving a backbone parameterization of the same order as Tucker.

Around this model, we further characterize several of its basic properties. First, NTD-PL and standard Tucker form a clear nesting relation: when the higher-order coefficients are turned off, the model exactly degenerates to Tucker; as the polynomial degree increases, the model class expands in a controlled way with degree. Second, when the observation tensor is generated by applying an element-wise polynomial function to some Tucker latent tensor, NTD-PL can achieve exact matching at the corresponding order; when the generating function is analytic, finite-order polynomial links provide systematic approximation with explicit truncation error bounds. Third, in order to handle both full-observation tensor decomposition and tensor completion with masked observations in a unified way, we construct a unified learning objective and combine coefficient ridge updates, factor normalization, and degree continuation to obtain a practical optimization framework.

Experimental results further support the above modeling and theoretical analysis. In controlled synthetic experiments, NTD-PL remains consistent with Tucker on strictly linear data, and under polynomial and smooth nonlinear residuals it continuously outperforms Tucker, CP, TT, and TR as nonlinear strength increases, with performance gains aligned with the dominant order of the target function. On real hyperspectral data, NTD-PL consistently outperforms Tucker on both full-observation reconstruction and random-missing completion on the CAVE dataset under the same parameter budget. These gains appear not only in scene-wise averages but also consistently across most scenes, multiple missing rates, and local spatial regions. Further mechanism analysis shows that this advantage cannot be attributed either to simply adding more parameters or to applying a single shared polynomial calibration to the Tucker output. The main contributions of this paper are summarized as follows.

1. We propose NTD-PL, a nonlinear low-rank tensor model that imposes an explicit element-wise polynomial link on a shared Tucker latent tensor.

2. We clarify its nesting relation with Tucker, its parameter-scale characteristics, and its expressivity under polynomial and analytic generation.
3. We provide a unified masked learning formulation so that the model can handle both full tensor decomposition and tensor completion.
4. We validate the effectiveness and applicability boundary of the model through controlled synthetic experiments and real hyperspectral experiments.

The remainder of the paper is organized as follows. Section 2 reviews related work. Section 3 introduces notation and preliminaries. Section 4 presents the model, its properties, and the optimization method. Section 5 reports the experimental results. Section 6 concludes the paper and discusses future work.

2 Related Work

From the perspective of the level at which nonlinearity is introduced into the model, existing nonlinear tensor decomposition methods can be roughly grouped into four categories.

The first category preserves the low-rank factor backbone while directly modifying the mapping from latent linear predictors to observed tensor entries, that is, it places nonlinearity at the entry-generation level. Early work in this direction extends classical decomposition models through generalized losses or generalized likelihoods. For example, generalized CP decomposition [11] preserves low-rank parameterization while replacing squared error with a more general observation loss, allowing the model to adapt to binary data, count data, and robust modeling scenarios. Bayesian Poisson Tucker decomposition [23] introduces a Poisson observation model and a hierarchical Bayesian structure into the Tucker framework for modeling count-valued multiway event data. Going further, InfTucker [31], Distributed Flexible Nonlinear Tensor Factorization [36], and Streaming Nonlinear Bayesian Tensor Decomposition [19] introduce stronger nonparametric entry-wise nonlinear expressivity through kernel methods, Gaussian processes, or their sparse approximations. Overall, this line of work is the closest to our paper, because they also aim to strengthen the observation-generation mechanism without completely abandoning the low-rank tensor backbone. The corresponding cost, however, is usually heavier inference, more kernel or variational hyperparameters, and relatively weaker structural transparency.

The second category places nonlinearity in the generation, constraint, or organization of the factors themselves, with the core idea being to change how the factors are constructed. Such methods typically bend the traditional linear factor space into a constrained nonlinear representation space through deep priors, temporal networks, or geometric manifold assumptions. For example, DeepTensor [22] generates low-rank factors by deep networks and uses the implicit regularization of deep priors to compensate for the limited expressivity of classical low-rank models on complex signal structures. Neural Tensor Factorization [29] and DeepCP [14] combine embeddings, temporal dependency, and neural generation mechanisms to handle temporal interactions and noisy observations. Neural Additive Tensor Decomposition [1] seeks a compromise between expressivity and interpretability by applying neural-network transformations to latent components so as to strengthen sparse tensor modeling. Other work emphasizes geometric structure more strongly, such as tensor decomposition with nonlinear manifold modeling for 3D head-pose estimation [8], where explicit manifold constraints on pose factors improve generalization and interpretability. Compared with entry-level nonlinearity, this category usually has greater modeling freedom, but it also depends more heavily on network architecture design or prior selection, and its theoretical analysis and optimization stability are more complicated.

The third category goes further by networkizing the tensor decomposition operator itself, that is, by directly embedding nonlinearity into mode products, core contractions, or hierarchical decomposition structures, thereby forming deep tensor models. The representative idea is not to attach a nonlinear component outside a traditional low-rank decomposition, but to reorganize Tucker, TT, and related structures into multi-layer, stackable nonlinear transformation units. For instance, Stacked Tucker Decomposition [30] inserts nonlinear activations after mode products and constructs a deep Tucker structure through hierarchical stacking. NMTucker [27] explicitly introduces nonlinear components into a Matryoshka-style Tucker framework to improve expressivity in highly sparse completion tasks. Nonlinear Tensor Train [28] inserts activations between TT-core contractions for deep network compression and representation learning. Compared with the previous two categories, the advantage of this line is that it can approach the expressive power of deep networks while preserving parameter sharing in tensor structures, but at the cost of weakening the directness of classical low-rank models in parameter meaning, structural interpretation, and theoretical characterization.

The fourth category does not necessarily modify tensor decomposition itself directly. Instead, it treats tensors as compact carriers for representing external nonlinear structures, and then uses decomposition for efficient modeling and computation. Such methods commonly arise in polynomial systems, Volterra systems, and nonlinear transform-domain representations, where the nonlinear structure is typically specified first by physical mechanisms, analytic expansions, or front-end transforms, and tensors are mainly responsible for compactly representing higher-order coefficients and accelerating numerical computation. Typical examples include TNMOR and STORM [7, 15], which first expand nonlinear circuits or dynamical systems via polynomial, Taylor, or Volterra series, then rearrange the coefficients of different orders into high-order tensors and apply CP or symmetric decomposition to construct reduced-order models that preserve higher-order nonlinearity. Another line of work, such as multidimensional nonlinear transform-domain tensor representation [24], builds data-adaptive nonlinear transform domains using convolutional networks and activation functions, and then imposes tensor low-rank constraints in that domain. Overall, these methods show that tensors can serve not only as nonlinear decomposers but also as compact representations of nonlinear structure. In most cases, however, the source of nonlinearity lies outside the tensor model rather than forming a controlled, explicit, and directly interpretable entry-level extension inside the classical Tucker backbone.

Compared with the above work, NTD-PL is positioned closer to an extension of the entry-generation mechanism that preserves the Tucker backbone, but it still differs from routes based on generalized likelihoods, kernel methods, Gaussian processes, and deep decoders. Our method does not rely on infinite-dimensional kernel mappings, heavy probabilistic inference, or deep generative networks. Instead, it introduces an explicit, finite-order, element-wise polynomial link on top of a shared Tucker latent tensor, so that nonlinear capability enters the model in a controlled way through only a few coefficients. This design allows NTD-PL to inherit the low-rank structure, parameter efficiency, and multiway interpretability of Tucker on the one hand, while on the other hand describing entry-level residuals through an explicit degree expansion and yielding a clear nesting relation with Tucker together with the corresponding approximation analysis. Relative to the existing literature, the novelty of NTD-PL lies precisely in the following point: it does not fully outsource nonlinearity to kernels, neural networks, or external polynomial systems, but instead constructs within the classical Tucker framework an explicit, analyzable, and degree-wise controllable nonlinear link mechanism, thereby striking a new balance among expressive power, structural transparency, and optimization practicality.

3 Notation and Preliminaries

We use calligraphic letters for tensors, bold uppercase letters for matrices, bold lowercase letters for vectors, and italic lowercase letters for scalars. Let

$$\mathcal{Y} \in \mathbb{R}^{I_1 \times \cdots \times I_N}$$

be an Nth-order observation tensor, and let $[I_n] = \{1, \dots, I_n\}$ denote the index set of mode n.

For tensors of the same size, $\odot$ denotes the Hadamard product. The element-wise power is written as $\mathcal{S}^{\odot p}$, with $(\mathcal{S}^{\odot p})_{\mathbf{i}} = s_{\mathbf{i}}^p$ and $\mathcal{S}^{\odot 0} = \mathbf{1}$. We use $\|\cdot\|_F$ for the Frobenius norm. The mode-n product between $\mathcal{X}$ and a matrix $\mathbf{A}$ is denoted by $\mathcal{X} \times_n \mathbf{A}$.

The Tucker model approximates $\mathcal{Y}$ as

$$\widehat{\mathcal{Y}} = [\![\mathcal{X}; \mathbf{A}^{(1)}, \dots, \mathbf{A}^{(N)}]\!] := \mathcal{X} \times_1 \mathbf{A}^{(1)} \times_2 \cdots \times_N \mathbf{A}^{(N)},$$

where $\mathcal{X} \in \mathbb{R}^{R_1 \times \cdots \times R_N}$ is the core tensor and $\mathbf{A}^{(n)} \in \mathbb{R}^{I_n \times R_n}$ are factor matrices.

To handle incomplete observations, we use a binary mask tensor $\Omega \in \{0, 1\}^{I_1 \times \cdots \times I_N}$. The observed-entry fitting form is then

$$\min_{\mathcal{X}, \{\mathbf{A}^{(n)}\}} \frac{1}{2} \left\| \Omega \odot \left(\mathcal{Y} - [\![\mathcal{X}; \mathbf{A}^{(1)}, \dots, \mathbf{A}^{(N)}]\!] \right) \right\|_F^2.$$

We also define an element-wise polynomial link. Given a degree P and coefficients $\boldsymbol{\beta} = (\beta_0, \beta_1, \dots, \beta_P)^\top$, let

$$f_{\boldsymbol{\beta}}(t) = \sum_{p=0}^{P} \beta_p t^p$$

for a scalar t. Applied element-wise to a tensor $\mathcal{S}$, this becomes

$$f_{\boldsymbol{\beta}}(\mathcal{S}) = \sum_{p=0}^{P} \beta_p \mathcal{S}^{\odot p}.$$

When $\beta_1 = 1$ and all other coefficients vanish, the link reduces to the identity map.

4 Proposed Method

Under the premise of preserving the Tucker low-rank backbone, how can we introduce higher-order interactions in a controlled way while still maintaining a manageable parameter scale, interpretability, and practical optimizability? To answer this question, this section first presents the structural motivation behind NTD-PL, then formally defines the model, summarizes its basic properties, and finally gives a unified learning objective and optimization framework consistent with the current implementation.

4.1 Design Motivation: Explicit Higher-Order Interactions

Standard Tucker decomposition $\mathcal{T}(\mathcal{X}, \{\mathbf{A}^{(n)}\})$ forms a multilinear generation mechanism through a shared core tensor and mode-wise factor matrices. Its essence is to generate the observation tensor through successive mode products in a low-rank latent space. This structure enjoys good parameter efficiency and structural transparency in compression, recovery, and representation learning, but it implicitly assumes that observed entries can be adequately described by a multilinear mechanism. For data with saturated responses, nonlinear observations, complex higher-order couplings, or curved structures, reliance on multilinearity alone often leads to systematic mismatch.

In the matrix setting, a natural enhancement is to generalize a linear mapping

$$\mathbf{y} = \mathbf{A}\mathbf{x} \tag{1}$$

to a mapping with quadratic interactions,

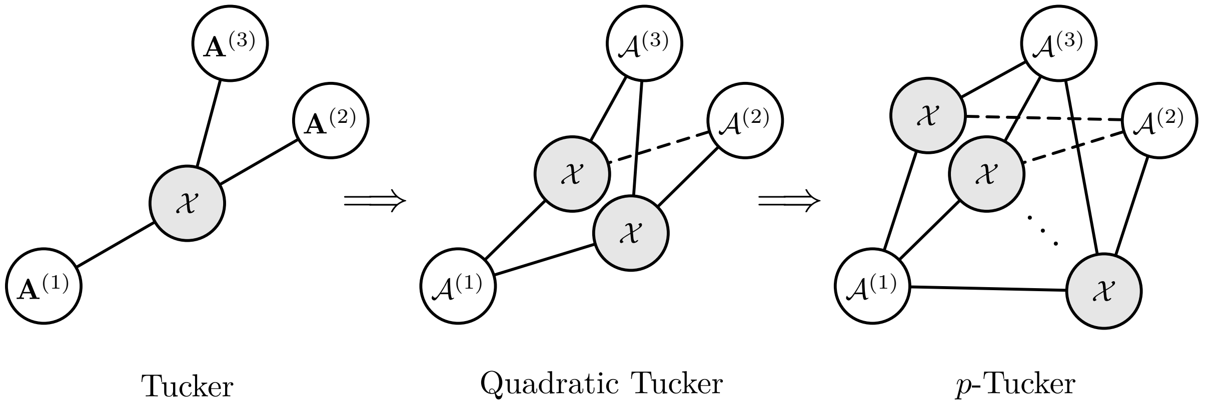
$$\mathbf{y} = \mathbf{A}\mathbf{x} + \mathcal{A}(\mathbf{x}, \mathbf{x}). \tag{2}$$

The central insight of QMF [34] is precisely that the role of the quadratic term is not simply to add parameters, but to provide the most basic step from locally flat representations to locally curved ones. Following the same line of thought, for tensors a natural question is: since Tucker is already the extension of linear low-rank models to multiway data, can we further bring explicit higher-order interactions into the generative mechanism of Tucker?

To this end, we write pth-order interactions into Tucker. A natural explicit prototype can be written as

$$\widehat{y}^{(p)}_{i_1\cdots i_N} = \sum_{\mathbf{r}^{(1)},\ldots,\mathbf{r}^{(p)}} \Big(\prod_{q=1}^{p} x_{\mathbf{r}^{(q)}}\Big) \prod_{n=1}^{N} a^{(n)}_{i_n, r_n^{(1)},\ldots,r_n^{(p)}}, \tag{3}$$

where each mode factor depends simultaneously on p latent indices in mode n. Equation (3) defines a higher-order Tucker decomposition: it preserves the mode-wise generative form of Tucker, but replaces the linear mapping in each mode with an explicit local coupling of order p. When $p = 2$, it corresponds to quadratic Tucker; for general p, it can be viewed as an explicit pth-order Tucker prototype, which we denote by p-Tucker. The following tensor network diagram gives a more intuitive illustration of this higher-order extension of Tucker.



Although this prototype is conceptually natural, it is not suitable as the final model. As the order p increases, the mode factors must be lifted to higher-order tensors, and the core tensor induces Kronecker-type higher-order expansions during generation. As a result, the parameter count, storage, and contraction cost all grow rapidly. More importantly, the optimization problem also becomes significantly harder. Therefore, explicit p-Tucker is better viewed as structural motivation: it explains how higher-order interactions may enter Tucker, but does not by itself provide a scalable implementation.

Based on this observation, instead of directly training the explicit higher-order model corresponding to (3), we seek a structured compression that preserves the semantics of higher-order interactions in p-Tucker while keeping the parameter scale of Tucker. NTD-PL is the model obtained under this design goal.

4.2 NTD-PL Model Definition and Structural Interpretation

4.2.1 Model Definition

We propose Nonlinear Tucker Decomposition with Polynomial Links, abbreviated as NTD-PL. Its core idea is to first generate a shared low-rank latent tensor by Tucker decomposition, and then apply an explicit element-wise polynomial link to this latent tensor. Specifically, let

$$\mathcal{S} = \mathcal{T}(\mathcal{X}, \{\mathbf{A}^{(n)}\}), \tag{4}$$

where $\mathcal{X} \in \mathbb{R}^{R_1\times\cdots\times R_N}$ is the core tensor and $\mathbf{A}^{(n)} \in \mathbb{R}^{I_n\times R_n}$ is the factor matrix for mode n. NTD-PL is defined as

$$\widehat{\mathcal{Y}} = f_{\boldsymbol{\beta}}(\mathcal{S}) = \sum_{p=0}^{P} \beta_p \mathcal{S}^{\odot p}, \tag{5}$$

where $\boldsymbol{\beta} = (\beta_0, \beta_1, \dots, \beta_P)^\top$ is the vector of polynomial link coefficients, and $\mathcal{S}^{\odot p}$ denotes the element-wise pth power of $\mathcal{S}$.

In (5), β_0 corresponds to a constant bias term, β_1 corresponds to the standard Tucker linear term, and β_p for $p \geq 2$ corresponds to higher-order interaction terms activated on the shared low-rank backbone. In particular, when

$$\beta_1 = 1, \qquad \beta_p = 0 \ (p \neq 1), \tag{6}$$

the model reduces exactly to standard Tucker. Therefore, NTD-PL is not another new backbone meant to replace Tucker, but a controlled nonlinear extension centered on the Tucker latent tensor.

NTD-PL should not be simply understood as performing Tucker reconstruction first and then applying an element-wise correction to the output. The key difference is that the nonlinearity in (5) acts not on an external signal independent of the low-rank structure, but on the shared Tucker latent tensor $\mathcal{S}$ itself. Since each entry of $\mathcal{S}$ is already generated through multilinear coupling between the core tensor and the mode factors, $\mathcal{S}^{\odot p}$ corresponds to a reorganization of interaction patterns on the same backbone, rather than an independent calibration applied after a completed linear reconstruction. Later, we will explicitly distinguish these two mechanisms by comparing polynomial post-processing of Tucker with NTD-PL.

4.2.2 A Structured Compression of p-Tucker

The definition of NTD-PL comes from a structured compression of p-Tucker. Take the quadratic case as an example. If we write the mode factor in (3) as

$$a^{(n)}_{i_n, r_n, r'_n} \approx b^{(n)}_{i_n, r_n} c^{(n)}_{i_n, r'_n}, \tag{7}$$

then the quadratic term can be rewritten as the element-wise product of two Tucker tensors:

$$\widehat{\mathcal{Y}}_2 \approx \mathcal{S}_B \odot \mathcal{S}_C, \tag{8}$$

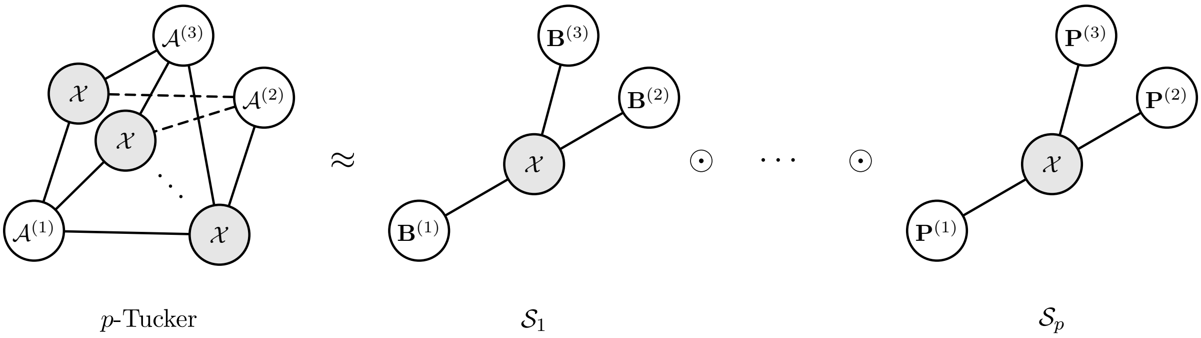
where

$$\mathcal{T}(\mathcal{X}, \{\mathbf{B}^{(n)}\}), \qquad \mathcal{T}(\mathcal{X}, \{\mathbf{C}^{(n)}\}). \tag{9}$$

If we further impose the shared projection constraint $\mathbf{B}^{(n)} = \mathbf{C}^{(n)}$, we obtain

$$\widehat{\mathcal{Y}}_2 \approx \mathcal{S}^{\odot 2}. \tag{10}$$

Extending the same idea to higher orders yields $\mathcal{S}^{\odot p}$ on a shared backbone. NTD-PL can therefore be understood as linearly combining these shared-projection pth-order interaction terms by their coefficients, thereby compressing the complexity of explicit p-Tucker into a small number of link coefficients. The following tensor network diagram illustrates this behavior more directly.



4.3 Basic Properties of NTD-PL

This subsection summarizes the three most basic properties of NTD-PL: model-family relations and parameter efficiency, expressivity under nonlinear generation, and scale indeterminacy together with the normalization mechanism. Detailed proofs are deferred to Appendix A–B. As we will see later, these properties are also validated in controlled synthetic experiments.

4.3.1 Model-Family Relations and Parameter Efficiency

Proposition 1 (Reduction to Tucker). *If $\beta_1 = 1$ and $\beta_p = 0$ for all $p \neq 1$, then (5) reduces to the standard Tucker model, namely*

$$\widehat{\mathcal{Y}} = \mathcal{S} = \mathcal{T}(\mathcal{X}, \{\mathbf{A}^{(n)}\}). \tag{11}$$

Proposition 2 (Degree Nesting). *For any $P_1 < P_2$, the degree-P_1 NTD-PL model class is a subset of the degree-P_2 model class.*

These two propositions show that NTD-PL forms a nested family of models with respect to the polynomial degree P: when the higher-order terms are turned off, it returns exactly to Tucker; as P increases, the model expressivity expands in a controlled manner by degree. This is also the theoretical basis for the degree-continuation optimization used later.

Proposition 3 (Parameter Complexity). *Let the multilinear rank be $(R_1, \ldots, R_N)$. Then the total number of parameters in NTD-PL is*

$$\#\text{param} = \prod_{n=1}^{N} R_n + \sum_{n=1}^{N} I_n R_n + (P+1), \tag{12}$$

where the first two terms correspond to the core and factor parameters of standard Tucker decomposition, and the last term corresponds to the polynomial link coefficients.

Proposition 3 shows that the additional degrees of freedom in NTD-PL are reflected only in $P+1$ scalar coefficients, rather than in new higher-order factor tensors or higher-order cores. Hence, compared with explicit p-Tucker, NTD-PL does not enhance expressivity by introducing another set of higher-order parameterizations, but by adding controlled nonlinearity while essentially preserving the parameter scale of Tucker.

4.3.2 Expressivity Under Nonlinear Generation

The expressivity of NTD-PL can be understood from the viewpoint of an element-wise nonlinear generation model. Suppose there exists a true Tucker latent tensor

$$\mathcal{S}^\star = \mathcal{T}(\mathcal{X}^\star, \{\mathbf{A}^{(n)\star}\}) \tag{13}$$

and suppose the observation tensor is generated by applying a scalar function $g : \mathbb{R} \to \mathbb{R}$ element-wise, namely

$$\mathcal{Y} = g(\mathcal{S}^\star). \tag{14}$$

Proposition 4 (Exact Representation Under Polynomial Generation). *If the observation tensor satisfies $\mathcal{Y} = g(\mathcal{S}^\star)$, where $g(s) = \sum_{p=0}^{P} a_p s^p$ is a polynomial of degree at most P, then NTD-PL can represent this generation process exactly by taking the latent tensor $\mathcal{S} = \mathcal{S}^\star$ and setting $\beta_p = a_p$. A detailed proof is given in Appendix Theorem 2.*

Theorem 1 (Explicit Truncation Error Bound for Analytic Functions). *Suppose g is analytic on the complex disk*

$$D(0,\rho) = \{z \in \mathbb{C} : |z| < \rho\} \tag{15}$$

and there exists a constant B such that

$$\|\mathcal{S}^\star\|_\infty \le B < \rho. \tag{16}$$

Let the Taylor expansion of g at 0 be

$$g(s) = \sum_{p=0}^{\infty} a_p s^p, \tag{17}$$

and let its degree-P truncated polynomial be

$$q_P(s) = \sum_{p=0}^{P} a_p s^p. \tag{18}$$

If we take the latent tensor of NTD-PL to be $\mathcal{S}^\star$ and set the link coefficients to $\beta_p = a_p$, then the approximation error satisfies

$$\|g(\mathcal{S}^\star) - q_P(\mathcal{S}^\star)\|_F \le \sqrt{N_e}\, M_\rho \frac{(B/\rho)^{P+1}}{1 - B/\rho}, \tag{19}$$

where

$$N_e = \prod_{n=1}^{N} I_n, \qquad M_\rho = \sup_{|z|=\rho} |g(z)|. \tag{20}$$

Equation (19) provides an error decomposition with a clear interpretation. First, the leading term $(B/\rho)^{P+1}$ shows that, when $B/\rho < 1$ is fixed, the truncation error decays geometrically with the polynomial degree P. Therefore, as long as the amplitude range of the true latent tensor does not approach the boundary of the radius of analyticity, finite-order NTD-PL can rapidly approximate the analytic generating function. Second, the denominator term $1 - B/\rho$ characterizes the margin to the nearest singularity: when B is smaller relative to ρ, the active range of the true signal lies deeper inside a stable analytic region, and the remainder bound becomes tighter; conversely, as B approaches ρ, approximation becomes substantially harder. Finally, $\sqrt{N_e}$ only reflects the scale enlargement that occurs when passing from uniform scalar approximation to the Frobenius error over the whole tensor, and does not change the convergence order with respect to P. Hence, the theorem shows that the approximation ability of NTD-PL is primarily determined by the analyticity of the link function over the effective input range and the relative magnitude of B/ρ, rather than by the introduction of additional higher-order tensor parameters. This will help explain the approximation behavior in Section 5.2.2.

4.3.3 Scale Indeterminacy and the Normalization Mechanism

Like standard Tucker, NTD-PL inherits the scale indeterminacy between the core tensor and the factor matrices. Since NTD-PL simultaneously combines $\mathcal{S}, \mathcal{S}^{\odot 2}, \ldots, \mathcal{S}^{\odot P}$, this scale issue is even more sensitive under a polynomial link: if the scale of $\mathcal{S}$ drifts, terms of different orders are amplified or shrunk at different rates, which directly affects the numerical stability of $\boldsymbol{\beta}$.

Proposition 5 (Scale Invariance). *Suppose an invertible diagonal scaling $D^{(n)}$ is applied to the factor matrix of some mode n, and the corresponding inverse scaling is simultaneously absorbed into the core tensor. Then the latent tensor $\mathcal{S}$ remains unchanged, and consequently the NTD-PL output $\widehat{\mathcal{Y}} = f_{\boldsymbol{\beta}}(\mathcal{S})$ also remains unchanged.*

Proposition 5 shows that column normalization of the factor matrices together with absorption of scale into the core tensor can be understood as a form of *gauge fixing*: it does not change the model class, but simply selects a more stable representative from an equivalence class of parameterizations. When higher-order terms participate in modeling, this step is particularly important for the estimation of $\boldsymbol{\beta}$, and is therefore included as a fixed step in the optimization algorithm below.

4.4 Unified Learning Objective and Optimization Algorithm

4.4.1 Unified Learning Objective

To handle decomposition and missing-data completion within a single optimization framework, let the binary observation mask tensor be denoted by $\Omega \in \{0,1\}^{I_1 \times \cdots \times I_N}$. Consider the following objective:

$$\min_{\mathcal{X},\{\mathbf{A}^{(n)}\},\boldsymbol{\beta}} \frac{1}{2Z_\Omega} \|\Omega \odot (\mathcal{Y} - f_{\boldsymbol{\beta}}(\mathcal{S}))\|_F^2 + \mathcal{R}(\mathcal{X}, \{\mathbf{A}^{(n)}\}, \boldsymbol{\beta}), \tag{21}$$

where $\mathcal{S} = \mathcal{T}(\mathcal{X}, \{\mathbf{A}^{(n)}\})$, and

$$Z_\Omega = \begin{cases} \prod_{n=1}^{N} I_n, & \Omega \equiv \mathbf{1}, \\ \sum\limits_{i_1,\dots,i_N} \Omega_{i_1 \cdots i_N}, & \text{otherwise}. \end{cases} \tag{22}$$

Here, Z_Ω normalizes the data-fitting term by the number of effective entries so that the gradient magnitude remains comparable between the full-observation and missing-observation cases. This is also consistent with the residual scaling used in the current implementation. The regularizer is taken as

$$\mathcal{R}(\mathcal{X}, \{\mathbf{A}^{(n)}\}, \boldsymbol{\beta}) = \frac{\lambda_X}{2}\|\mathcal{X}\|_F^2 + \sum_{n=1}^{N} \frac{\lambda_n}{2}\|\mathbf{A}^{(n)}\|_F^2 + \frac{\lambda_\beta}{2}\|\boldsymbol{\beta}\|_2^2. \tag{23}$$

When $\Omega \equiv \mathbf{1}$, (21) becomes the NTD-PL decomposition objective for a fully observed tensor; when Ω is a general mask, it becomes a completion objective trained only on the observed entries. In this way, decomposition and completion no longer require two separate models; they are simply two observation settings under the same learning objective.

4.4.2 Initialization

As a polynomial extension of Tucker, NTD-PL can naturally use Tucker decomposition as the optimization starting point. For fully observed data we use standard Tucker initialization, and for missing observations we use masked Tucker initialization [16, 32]. For the polynomial coefficients, we use a linear warm start:

$$\beta_0 = 0, \qquad \beta_1 = 1, \qquad \beta_p = 0 \quad (p \geq 2). \tag{24}$$

We adopt a continuation-style optimization strategy, starting from the linear case $p = 1$ and progressively activating higher-order terms as optimization proceeds. This means that the model always starts from a neighborhood of the standard Tucker solution, rather than from a fully nonlinear high-order model from the outset.

4.4.3 Updates for the Core and the Factors

After fixing $\boldsymbol{\beta}$, we first construct the latent tensor

$$\mathcal{S} = [\![\mathcal{X}; \mathbf{A}^{(1)}, \dots, \mathbf{A}^{(N)}]\!], \tag{25}$$

and compute the prediction tensor

$$\widehat{\mathcal{Y}} = f_{\boldsymbol{\beta}}(\mathcal{S}), \qquad f'_{\boldsymbol{\beta}}(\mathcal{S}) = \sum_{p=1}^{P} p\beta_p \mathcal{S}^{\odot(p-1)}. \tag{26}$$

In the current implementation, $f_{\boldsymbol{\beta}}(\mathcal{S})$ and $f'_{\boldsymbol{\beta}}(\mathcal{S})$ are evaluated simultaneously through a fused Horner recursion to reduce repeated power computations and memory access. This only changes the constant factor in runtime and does not change the model itself.

Let the normalized masked residual be

$$\mathbf{E} = \frac{1}{Z_\Omega}\,\Omega \odot (\widehat{\mathcal{Y}} - \mathcal{Y}), \tag{27}$$

and define

$$\mathbf{T} = \mathbf{E} \odot f'_{\boldsymbol{\beta}}(\mathcal{S}). \tag{28}$$

Then, by the chain rule, the gradient of the objective with respect to the latent tensor $\mathcal{S}$ is proportional to $\mathbf{T}$. Propagating further through the Tucker reconstruction relation gives the core-tensor gradient

$$\nabla_{\mathcal{X}}\mathcal{L} = \mathbf{T} \times_1 \mathbf{A}^{(1)\top} \times_2 \cdots \times_N \mathbf{A}^{(N)\top} + \lambda_X \mathcal{X}. \tag{29}$$

For the factor matrix in mode n, let

$$\mathbf{Z}^{(n)} = \Big[\mathcal{X} \times_{m\neq n} \mathbf{A}^{(m)}\Big]_{(n)}, \tag{30}$$

where $[\cdot]_{(n)}$ denotes mode-n unfolding. Then the corresponding gradient can be written as

$$\nabla_{\mathbf{A}^{(n)}}\mathcal{L} = \mathbf{T}_{(n)}\,\mathbf{Z}^{(n)\top} + \lambda_n \mathbf{A}^{(n)}. \tag{31}$$

Since the element-wise polynomial link breaks the closed-form least-squares structure of classical Tucker subproblems, we do not use alternating least squares. Instead, we update $\mathcal{X}$ and all factor matrices by Adam [12].

4.4.4 Update of the Polynomial Coefficients

When $\mathcal{X}$ and $\{\mathbf{A}^{(n)}\}_{n=1}^{N}$ are fixed, the latent tensor $\mathcal{S}$ is also fixed. At this point, the update of $\boldsymbol{\beta}$ reduces to a low-dimensional ridge-regression problem [10]. Let the observation vector be $\mathbf{y}_\Omega$, and define the design matrix

$$\Phi_\Omega = \Big[\mathbf{1}, \mathrm{vec}(\mathcal{S})_\Omega, \mathrm{vec}(\mathcal{S}^{\odot 2})_\Omega, \ldots, \mathrm{vec}(\mathcal{S}^{\odot P})_\Omega\Big] \in \mathbb{R}^{|\Omega|\times(P+1)}. \tag{32}$$

Then the $\boldsymbol{\beta}$ subproblem is

$$\min_{\boldsymbol{\beta}} \frac{1}{2}\|\mathbf{y}_\Omega - \Phi_\Omega\boldsymbol{\beta}\|_2^2 + \frac{\lambda_\beta}{2}\|\boldsymbol{\beta}\|_2^2. \tag{33}$$

If the constant term is allowed to participate in fitting, we solve (33) directly. If not, we fix $\beta_0 = 0$ and perform the same ridge update only for the coefficients with $p \geq 1$. Since $\lambda_\beta > 0$, the system matrix $\Phi_\Omega^\top\Phi_\Omega + \lambda_\beta I$ is always positive definite. Therefore, this subproblem is strongly convex and has a unique solution. A strict proof is given in Appendix C.1.

Although the overall problem is nonconvex jointly in $(\mathcal{X}, \{\mathbf{A}^{(n)}\}, \boldsymbol{\beta})$, once the Tucker latent variable $\mathcal{S}$ is fixed, the update of $\boldsymbol{\beta}$ is always a strongly convex ridge-regression problem of size only $P+1$. This is one of the key reasons why the paper can retain higher-order nonlinear expressivity while still using a stable alternating optimization framework.

Power bases of high order may be numerically ill-conditioned, especially when the range of $\mathcal{S}$ is large. To improve this, in implementation we may first solve for coefficients $\boldsymbol{\gamma}$ on the power basis of the standardized variable

$$\mathbf{Z} = \frac{\mathcal{S} - \mu}{\sigma}, \tag{34}$$

Algorithm 1 NTD-PL with alternating optimization and degree continuation

```
Input: observation tensor 𝒴, mask Ω, multilinear rank {R_n}, maximum polynomial degree P
Initialize (𝒳, {A^(n)}) by Tucker / masked Tucker
Initialize β_0 = 0, β_1 = 1, β_p = 0 (p ≥ 2)
If continuation is enabled, set the current degree to p ← 1; otherwise set it to p ← P
for t = 1, 2, ..., T do
    If a new degree milestone is reached, activate the next order and set the new coefficient to zero
    Construct the latent tensor 𝒮 = ⟦𝒳; A^(1), ..., A^(N)⟧
    Compute f_β(𝒮) and f'_β(𝒮), and obtain the residual tensor and the intermediate tensor T
    Update the core tensor 𝒳 and the mode factors {A^(n)} by Adam
    Normalize the factor columns and absorb their scales into the core tensor
    if t reaches a β update interval then
        Fix the current 𝒮 and update β by ridge regression
    end if
end for
Output: 𝒳, {A^(n)}, and β
```

and then map them back to the original power-basis coefficients $\boldsymbol{\beta}$ by a basis transformation. This process only improves numerical conditioning in solving the subproblem and does not change the form of the outer model.

4.4.5 Factor Normalization and Core Absorption

After each round of gradient updates, we normalize every column of each factor matrix to unit ℓ_2 norm and absorb the corresponding scale into the core tensor. According to Proposition 5, this step does not change the latent tensor $\mathcal{S}$, and therefore does not change the output of NTD-PL. Its role is to reduce scale drift caused by equivalent parameterizations, especially to weaken the coupling effect of scale variation in $\mathcal{S}$ on higher-order terms $\mathcal{S}^{\odot p}$ and on the estimation of $\boldsymbol{\beta}$.

4.4.6 Degree Continuation Strategy

Directly optimizing a high-order polynomial model from scratch is often unstable. For this reason, we adopt a degree continuation strategy: training starts from the linear Tucker case $p = 1$, and higher-order terms are gradually activated at preset milestones $\tau_2 < \cdots < \tau_P$. Whenever the pth-order term is activated, the corresponding new coefficient is initialized to zero, namely

$$\boldsymbol{\beta}^{\text{new}} = (\beta_0, \ldots, \beta_{p-1}, 0)^\top. \tag{35}$$

In this way, the model transitions gradually from a stable linear solution to a richer nonlinear one, rather than jointly optimizing all coefficients in a high-order search space from the beginning. Proposition 2 has already shown that such continuation is not merely a heuristic trick, but a natural optimization path that follows the nested structure of the model family.

4.4.7 Complexity Analysis of the Algorithm

Let the total number of entries in the observation tensor be

$$M = \prod_{n=1}^{N} I_n, \qquad M_\Omega = |\Omega|, \qquad R = \prod_{n=1}^{N} R_n. \tag{36}$$

Here, M denotes the total number of entries in the full tensor, and M_Ω denotes the number of observed entries.

For one standard Tucker reconstruction, if implemented by successive mode products, the forward complexity can be written as

$$\mathcal{C}_{\text{rec}} = \sum_{n=1}^{N} I_n R_n \left(\prod_{m<n} I_m \right) \left(\prod_{m>n} R_m \right). \tag{37}$$

This term gives the main cost of constructing the latent tensor $\mathcal{S}$, and also serves as the basis for subsequent gradient propagation.

Given $\mathcal{S}$, the costs of computing $f_{\boldsymbol{\beta}}(\mathcal{S})$ and $f'_{\boldsymbol{\beta}}(\mathcal{S})$ are both element-wise polynomial evaluations. If a fused Horner recursion is used, they can be completed simultaneously in a single pass, with asymptotic complexity

$$\mathcal{O}(PM). \tag{38}$$

Compared with computing the forward value and derivative separately, this fused implementation only reduces the constant factor, without changing the order with respect to P and M.

The computation of the core gradient can be viewed as projecting the tensor $\mathbf{T}$ back into the core space, with complexity of the same order as one Tucker compression. The computation of the factor gradients has an MTTKRP-like structure: for each mode n, we first construct $\mathbf{Z}^{(n)}$ in (30), and then compute $\mathbf{T}_{(n)}\mathbf{Z}^{(n)\top}$. Therefore, if each outer iteration performs K Adam steps on $(\mathcal{X}, \{\mathbf{A}^{(n)}\})$, the dominant cost can be approximated as

$$\mathcal{O}\big(K(\mathcal{C}_{\text{rec}} + PM + \mathcal{C}_{\text{bwd}})\big), \tag{39}$$

where $\mathcal{C}_{\text{bwd}}$ represents the backward cost composed of the core back-projection and the MTTKRP-like multiplications for all mode factors, and is typically of the same order as the contraction/unfolding complexity of the Tucker backbone itself.

When $\mathcal{S}$ is fixed, the $\boldsymbol{\beta}$ subproblem depends only on the observed entries. Constructing the observed design matrix $\Phi_\Omega \in \mathbb{R}^{M_\Omega \times (P+1)}$ costs

$$\mathcal{O}(PM_\Omega), \tag{40}$$

and solving the normal equation

$$(\Phi_\Omega^\top \Phi_\Omega + \lambda_\beta I)\boldsymbol{\beta} = \Phi_\Omega^\top \mathbf{y}_\Omega \tag{41}$$

costs

$$\mathcal{O}(M_\Omega P^2 + P^3). \tag{42}$$

Since P is usually small in the settings of this paper, such as $2, 3, 4$, the update of $\boldsymbol{\beta}$ is theoretically an extra step but is usually not the dominant component of the total runtime.

The cost of factor normalization and core absorption is

$$\mathcal{O}\left(\sum_{n=1}^{N} I_n R_n + R \right), \tag{43}$$

which is typically negligible compared with (39). Degree continuation does not change the asymptotic complexity of each individual outer iteration; it only affects the total number of iterations by gradually activating higher-order terms. In implementation, avoiding repeated Tucker reconstruction, fusing polynomial and derivative evaluation, and solving for $\boldsymbol{\beta}$ stably on a standardized power basis all reduce only constant costs or improve numerical conditioning, without changing the above asymptotic orders.

Putting the above pieces together, the complexity of one outer iteration of NTD-PL under the current dense implementation can be summarized as

$$\mathcal{O}\Big(K(\mathcal{C}_{\text{rec}} + PM + \mathcal{C}_{\text{bwd}}) + PM_\Omega + M_\Omega P^2 + P^3\Big). \tag{44}$$

Therefore, compared with standard Tucker, the extra computation of NTD-PL mainly comes from two parts: first, the computation of the element-wise polynomial link and its derivative, whose cost is the low-order, parallelizable term $\mathcal{O}(PM)$; second, the small-scale ridge update of $\boldsymbol{\beta}$, whose cost is $\mathcal{O}(PM_{\Omega} + M_{\Omega}P^2 + P^3)$. In other words, under the current implementation, the dominant cost still comes from the forward and backward propagation of the Tucker backbone itself, rather than from solving the polynomial link coefficients.

5 Experiments

This section reports the experimental results of NTD-PL on synthetic data and real hyperspectral data under a unified protocol. The experiments proceed through shared settings, controlled synthetic data, and recovery and completion on real hyperspectral data, with the goal of clarifying the scope of applicability of the method, the source of its gains, and its computational cost.

5.1 Experimental Setup

This subsection gives the data-generation principles, comparison protocol, and evaluation metrics shared by the subsequent experiments. Except for implementation details specific to individual subsections, all results follow the same reproducible experimental protocol.

To define nonlinear strength consistently across different nonlinear mappings, we define α as the energy ratio of the orthogonal nonlinear residual, rather than as the amplitude coefficient placed directly in front of the nonlinear function. Let the linear low-rank backbone be

$$\mathcal{S}^{\star} \in \mathbb{R}^{I_1 \times \cdots \times I_N}, \tag{45}$$

and given an element-wise mapping $g(\cdot)$, we first construct the orthogonal residual relative to $\mathcal{S}^{\star}$:

$$\mathcal{R}^{\star} = g(\mathcal{S}^{\star}) - \frac{\langle g(\mathcal{S}^{\star}), \mathcal{S}^{\star} \rangle}{\|\mathcal{S}^{\star}\|_F^2} \mathcal{S}^{\star}, \tag{46}$$

and then normalize it as

$$\widetilde{\mathcal{R}}^{\star} = \frac{\|\mathcal{S}^{\star}\|_F}{\|\mathcal{R}^{\star}\|_F} \mathcal{R}^{\star}. \tag{47}$$

The final observation tensor is defined by

$$\mathcal{Y}_{\alpha} = \sqrt{1-\alpha}\, \mathcal{S}^{\star} + \sqrt{\alpha}\, \widetilde{\mathcal{R}}^{\star}, \qquad 0 \le \alpha \le 1. \tag{48}$$

Thus, α represents the proportion of observation energy allocated to the orthogonal nonlinear residual. As α increases, the observation progressively deviates from the linear low-rank backbone, rather than merely increasing in amplitude. The `affine shift control` appearing in the linear-consistency subsection is essentially the addition of a constant bias to linear Tucker source data followed by energy normalization.

The controlled synthetic experiments consider two polynomial residual families and two smooth residual families:

$$g_{\text{poly2}}(s) = s^2, \qquad g_{\text{poly3}}(s) = s^2 + s^3, \tag{49}$$

$$g_{\sin}(s) = \sin(\kappa_{\sin} s), \qquad g_{\tanh}(s) = \tanh(\kappa_{\tanh} s). \tag{50}$$

Here, $\kappa_{\sin}$ and $\kappa_{\tanh}$ are set adaptively according to the input scale so that the functions do not stay in the linear regime for too long or saturate too early.

NMSE and RMSE are used as the main metrics. In the real hyperspectral experiments, full-observation reconstruction is uniformly reported using RMSE, NMSE, and SAM; random-missing completion is primarily reported using RMSE*, NMSE*, and SAM*. Metrics marked with * are computed only on genuinely observed missing entries and therefore serve as the primary indicators of completion quality.

For the percentage contribution of the β_p terms discussed later, we uniformly use

$$c_p = \frac{\|\beta_p \mathcal{S}^{\odot p}\|_F}{\sum_{q=0}^{P} \|\beta_q \mathcal{S}^{\odot q}\|_F} \times 100\%. \tag{51}$$

This definition characterizes the relative strength allocation of the terms of different orders in the overall expansion, and the values $\{c_p\}$ sum to exactly 100% over p.

The linear low-rank baselines compared in this paper include Tucker, CP, TT, and TR. Tucker decomposition is the most direct linear reference for our method. CP decomposition represents a tensor as a sum of rank-one tensors; it has a more compact parameterization and is also one of the most classical baselines in low-rank tensor modeling [13]. TT decomposition uses a chain-structured core factorization for high-order tensors and can significantly reduce storage and computational cost in high-order settings [18]. TR decomposition further closes the chain structure into a ring, making rank allocation more flexible than in TT [35]. Note that these decompositions differ in structure, and their notions of rank are not the same, so exact parameter matching is impossible. In the experiments, we choose ranks so that the parameter counts are as similar as possible.

5.2 Controlled Synthetic Data Experiments

This subsection compares linear baselines and NTD-PL under controlled mechanisms. The experiments contain two parts: first, on linear source data, we compare the paired errors of Tucker and linearly restricted NTD-PL; second, under a unified nonlinear residual protocol, we compare how the errors of different methods vary with α and P.

5.2.1 Linear-Consistency Verification

On linear source data, we compare the paired errors of Tucker and NTD-PL to check whether the latter reduces exactly to Tucker. Specifically, the data are Tucker tensors generated under multiple random seeds, with rank $[4, 4, 4]$, together with an affine bias `bias`, followed by energy normalization and additive noise at 30 dB. For NTD-PL, we set $P = 6$ and optionally allow or disallow β_0.

Table 1 shows that when `bias=0`, the fitting errors of Tucker and NTD-PL without a constant term almost completely overlap, and the higher-order contributions are very small, confirming the linear consistency of NTD-PL. When `bias=0.5`, NTD-PL can still fit the data well, whereas Tucker and NTD-PL with restricted β_0 exhibit systematic bias. Interestingly, one can observe that when β_0 is restricted, NTD-PL automatically uses higher-order terms to fit the bias, and the result is still slightly better than Tucker. Further, the distribution of paired differences is shown in Appendix Figure 11, where it is even more visually clear that when no affine bias is present, the difference between Tucker and NTD-PL is concentrated around zero, whereas in the presence of bias a more obvious systematic paired gap appears between the restricted model and Tucker.

5.2.2 Experiments on Nonlinear Mapping Matching and Approximation

Under the unified nonlinear generation protocol described in Section 5.1, we consider both polynomial and general nonlinear cases. Specifically, starting from Tucker tensors generated under multiple random seeds, we apply four classes of element-wise nonlinear residuals, namely `poly2`, `poly3`, `sin`, and `tanh`, and control the energy ratio of the nonlinear residual by (48). The experiment uniformly scans $\alpha \in \{0.10, 0.15, 0.20, 0.25, 0.30, 0.35, 0.40\}$ and $P \in \{1, 2, 3, 4, 5, 6\}$,

Table 1: Comparison of Tucker and NTD-PL on purely linear source data, used to verify the linear consistency of NTD-PL.

Setting	Method	RMSE	NMSE(dB)	$p > 1$ contrib. (%)
bias=0	Tucker	0.011398 ± 0.000747	-38.8822 ± 0.5704	—
	NTD-PL ($\beta_0 = 0$)	0.011610 ± 0.000658	-38.7173 ± 0.4883	4.19 ± 2.80
	NTD-PL (β_0 free)	0.011698 ± 0.000634	-38.6500 ± 0.4650	3.19 ± 2.15
bias=0.5	Tucker	0.064650 ± 0.016502	-24.0505 ± 2.0996	—
	NTD-PL ($\beta_0 = 0$)	0.052204 ± 0.011788	-25.8466 ± 1.8304	39.17 ± 11.56
	NTD-PL (β_0 free)	0.011797 ± 0.000596	-38.5755 ± 0.4319	2.98 ± 2.00

and compares the NMSE performance of Tucker, CP, TT, TR, and NTD-PL over 10 random seeds under similar parameter budgets.

Figure 1 shows that under all types of nonlinear residuals, NTD-PL maintains a stable lead over the entire range of nonlinear strength, indicating its advantage in handling nonlinear residuals. In addition, the approximation error of NTD-PL can be seen to scale with α, and the trend is consistent with the appendix discussion on the relation between truncation error and α: under the generation protocol used in this paper, the greater the energy proportion of the nonlinear residual, the larger the orthogonal residual that a finite-order polynomial link needs to absorb. Therefore, the error curve rises overall as α increases, but the advantage of NTD-PL relative to linear baselines is retained.

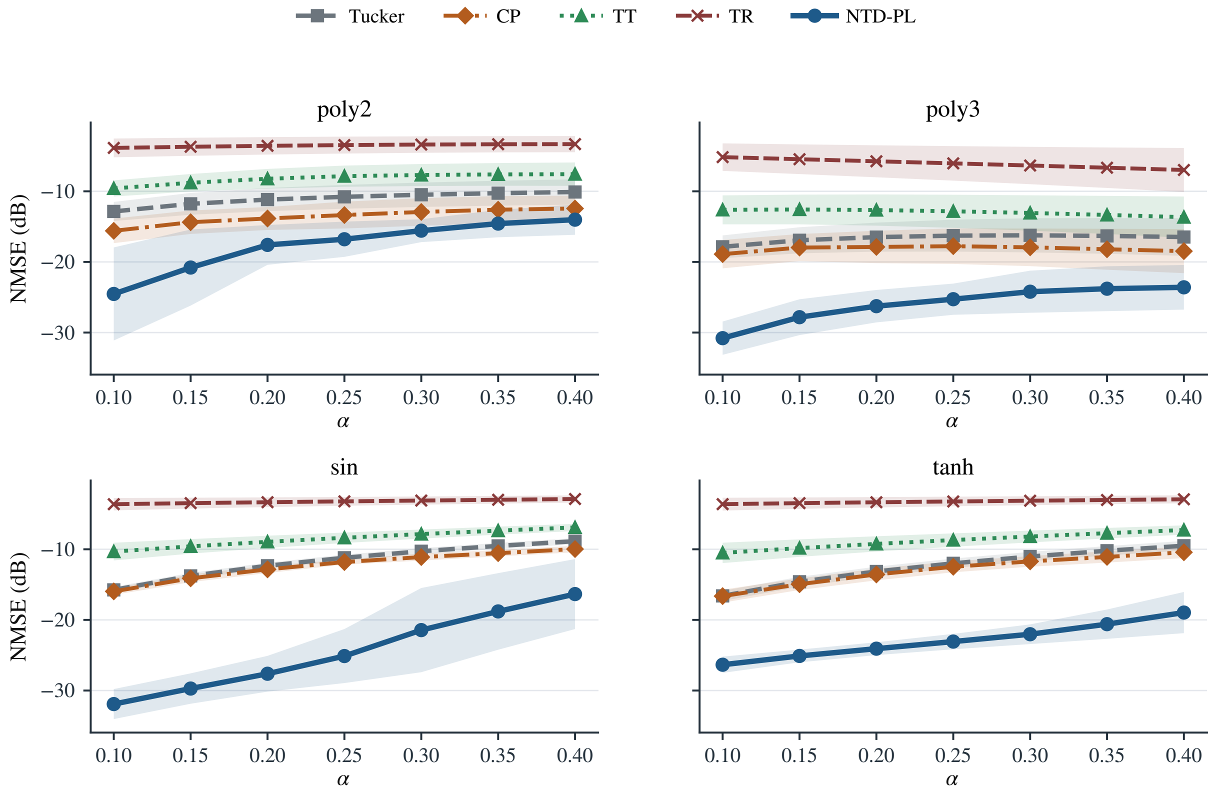


Figure 1: Approximation error of NTD-PL and the baseline methods as a function of nonlinear strength.

Table 2: Contribution of each degree term in NTD-PL (in %).

nonlinear($\alpha = 0.25$)	β_0	β_1	β_2	β_3	β_4	β_5	β_6
poly2	0.5 ± 0.8	43.3 ± 17.5	23.2 ± 6.9	5.8 ± 4.9	8.2 ± 5.8	9.1 ± 9.5	9.8 ± 9.7
poly3	0.1 ± 0.1	24.1 ± 6.5	9.8 ± 4.1	38.1 ± 9.0	7.3 ± 5.5	11.5 ± 11.1	9.1 ± 6.3
sin	0.1 ± 0.1	37.7 ± 8.8	0.6 ± 0.6	35.4 ± 3.5	2.8 ± 2.7	19.8 ± 6.9	3.6 ± 3.8
tanh	0.2 ± 0.1	37.9 ± 9.0	1.3 ± 0.7	30.9 ± 4.0	5.4 ± 3.7	17.3 ± 3.5	7.0 ± 5.8

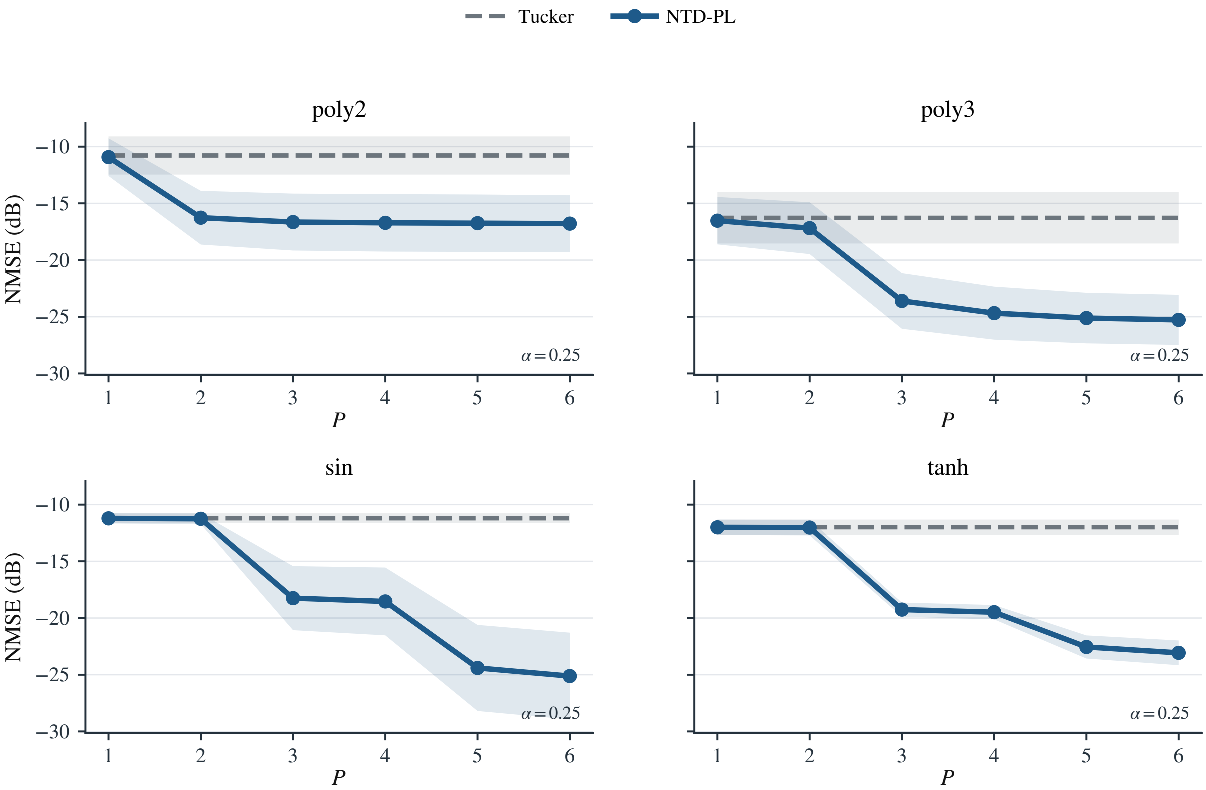


Figure 2: Approximation error of NTD-PL as a function of the degree P.

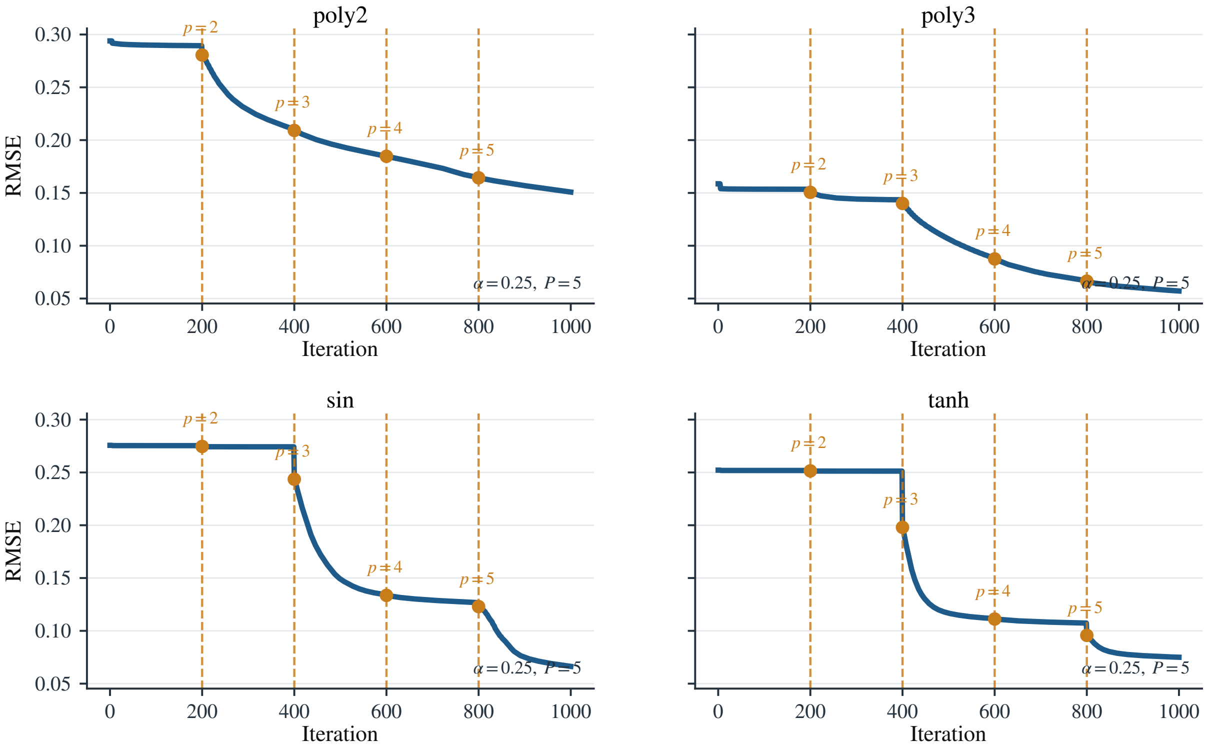


Figure 3: Training-loss curves of NTD-PL under the progressive degree-increasing strategy.

5.3 Nonlinear Low-Rank Reconstruction and Completion on Hyperspectral Data

We now turn to experimental validation on real hyperspectral data. We focus on answering three questions. First, does the overall gain of NTD-PL over linear Tucker still hold in real scenes? Second, where exactly does this gain come from? Third, does this advantage exhibit clear spatial localization, degree-related mechanisms, and cross-dataset stability? To answer these questions, the remainder of this section proceeds through six aspects: full-observation reconstruction, random-missing completion, mechanism discrimination, localization of local gains, analysis of degree saturation, and cross-dataset robustness, thereby forming a complete chain of evidence from overall performance to mechanism interpretation to external validation.

5.3.1 Data and Protocol

The experiments are organized into two layers: primary mechanism analysis and external robustness validation. The main experiments use 15 scenes from the CAVE dataset [3, 33] to systematically analyze the overall gains of NTD-PL on real hyperspectral tensors, the spatial distribution of local gains, and the mechanism related to polynomial degree. The external validation further uses four commonly used benchmark scenes, Jasper Ridge [21], Samson [4], Urban [9], and Cuprite [20], to test whether the main conclusions continue to hold across different spatial sizes, numbers of spectral bands, and scene properties.

All real-data experiments are conducted under full observation, with the original data size preserved, no additional cropping or downsampling, and no special selection of particular scenes. In preprocessing, we uniformly use global max normalization. The evaluation metrics follow Section 5.1: full-observation reconstruction reports RMSE, NMSE(dB), and SAM; random-missing completion uses RMSE*, NMSE(dB)*, and SAM*, computed only on the missing entries, as the main metrics. For the completion experiments, Tucker and NTD-PL share the same observation mask under the same scene, missing rate, and random seed, so as to ensure a fair paired comparison.

For parameters, the main text reports three rank budgets for CAVE full-observation reconstruction, namely $(24, 24, 4)$, $(33, 33, 4)$, and $(40, 40, 5)$, in order to examine whether the trend remains stable under different compression levels. For CAVE random-missing completion, we uniformly consider four missing rates, $\rho = 0.1, 0.3, 0.5, 0.7$, and compare Tucker and NTD-PL under the same rank setting and the same training protocol. The later mechanism comparison, hard-pixel / boundary-region analysis, and degree-mechanism analysis are all built on this unified completion protocol, so as to avoid introducing additional confounding factors through changes in task setup.

The cross-dataset robustness validation no longer uses one fixed rank. Instead, for each dataset, a rank setting is automatically determined by a unified compression-ratio matching rule. For these additional datasets, the completion task uniformly uses a random element-wise missing rate of $\rho = 0.5$. Therefore, the cross-dataset comparison in the main text is not based on individually tuned best-case results for each dataset, but is an external validity test across different real scenes under a unified protocol.

5.3.2 Full-Observation Reconstruction

We first examine low-rank reconstruction performance under full observation. Table 3 reports the scene-wise mean results under the three parameter budgets. It can be seen that under all three settings, $(24, 24, 4)$, $(33, 33, 4)$, and $(40, 40, 5)$, NTD-PL achieves lower RMSE, lower NMSE(dB), and lower SAM than Tucker. This shows that polynomial links bring stable gains while keeping the parameter scale of the Tucker low-rank backbone unchanged.

Table 3: Full-observation low-rank reconstruction results on CAVE. Under all three rank settings, NTD-PL and Tucker are compared under the same parameter budget, and NTD-PL achieves lower RMSE, NMSE(dB), and SAM.

Rank	Method	Param.	RMSE↓	NMSE(dB)↓	SAM(°)↓
$(24, 24, 4)$	Tucker	27,004	0.0299 ± 0.0198	-17.610 ± 4.498	17.42 ± 7.42
	NTD-PL	27,011	**0.0256 ± 0.0178**	**-19.058 ± 4.725**	**15.15 ± 6.40**
$(33, 33, 4)^{\star}$	Tucker	38,272	0.0261 ± 0.0172	-18.801 ± 4.432	16.43 ± 7.22
	NTD-PL	38,279	**0.0223 ± 0.0150**	**-20.125 ± 4.515**	**14.41 ± 6.30**
$(40, 40, 5)$	Tucker	49,115	0.0217 ± 0.0161	-20.739 ± 5.089	12.65 ± 5.93
	NTD-PL	49,122	**0.0192 ± 0.0135**	**-21.509 ± 4.540**	**11.85 ± 5.84**

$\star$ Main report rank in Sec. 5.3.2.

Table 4: Scene-level comparison for full-observation reconstruction on CAVE. NTD-PL outperforms Tucker on the vast majority of scenes, indicating that the overall gain is not driven by only a few anomalous scenes.

Metric	Win/Loss/Tie	Median gain	Sign test p	Wilcoxon p
RMSE	14/1/0	0.0028	9.77×10^{-4}	1.22×10^{-4}
NMSE_dB	14/1/0	1.1727	9.77×10^{-4}	1.22×10^{-4}
SAM	13/2/0	1.6423	0.007	0.022

5.3.3 Random-Observation Completion

Under random missing observations, we further examine the behavior of NTD-PL in recovering missing entries. Table 5 gives the main results under four missing rates. It can be seen that

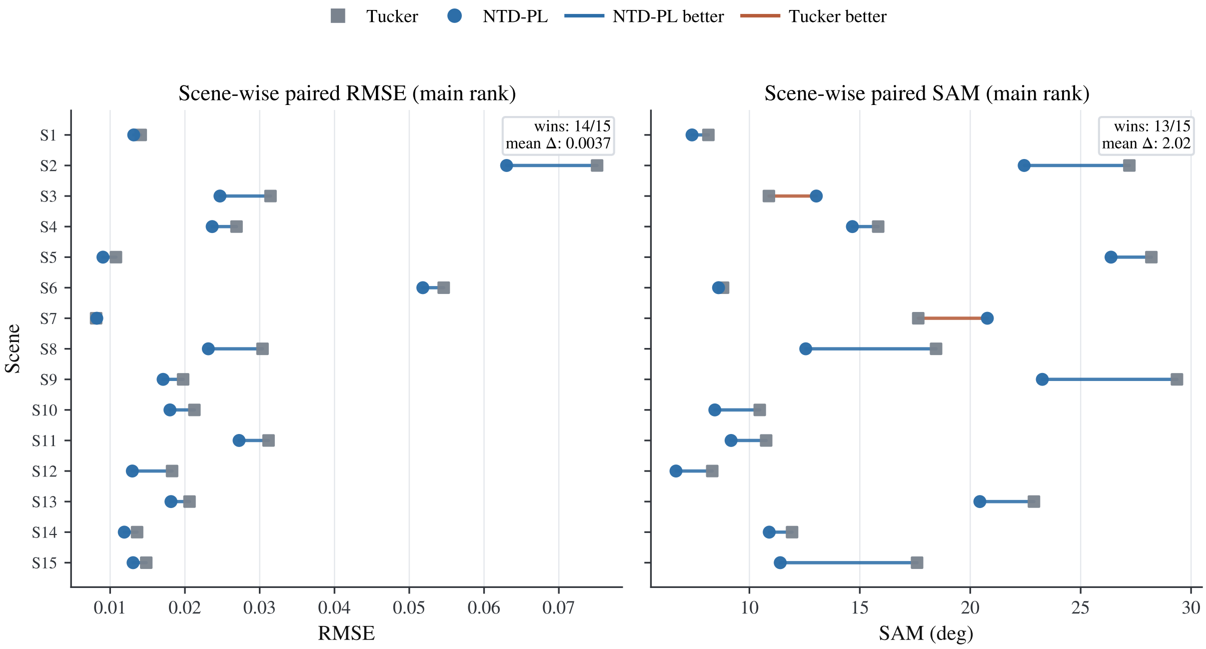


Figure 4: Scene-level visualization of full-observation reconstruction on CAVE. Each line connects the Tucker and NTD-PL results for the same scene. Overall, most scenes shift toward the side where NTD-PL is better, indicating consistency of the advantage at the scene level.

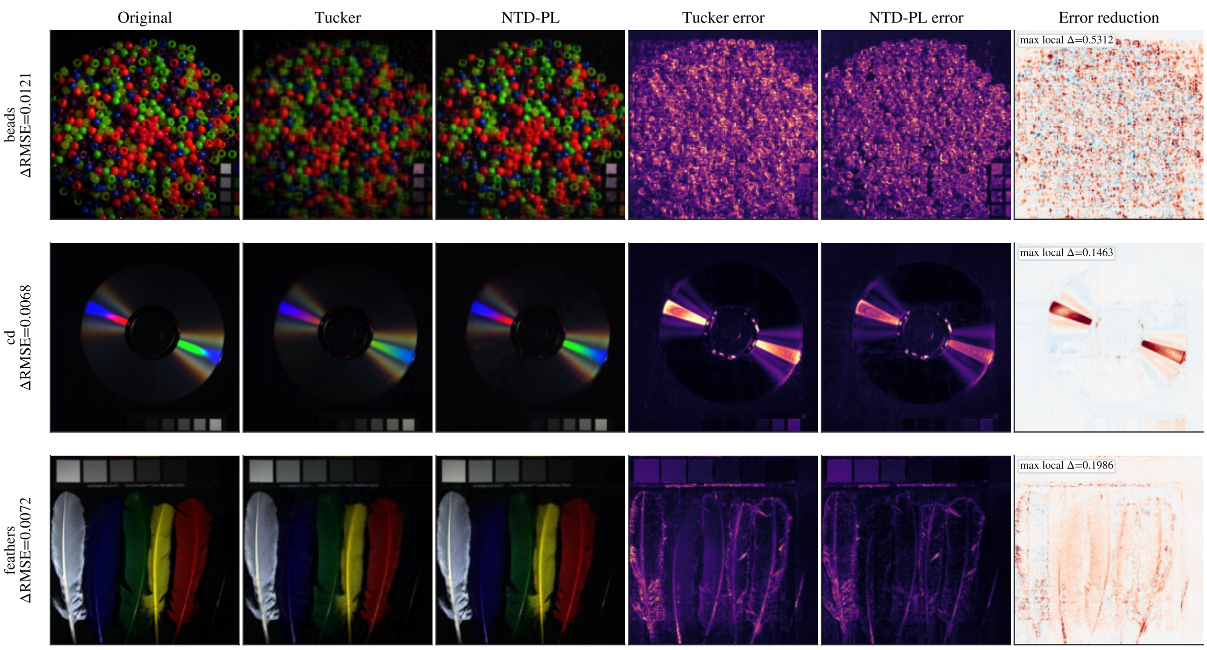


Figure 5: Spatial visualization of full-observation reconstruction on CAVE. Each row corresponds to a representative scene and shows the original image, reconstruction, pixel-wise error map, and error-reduction map. In the last column, positive values indicate that NTD-PL further reduces the reconstruction error relative to Tucker.

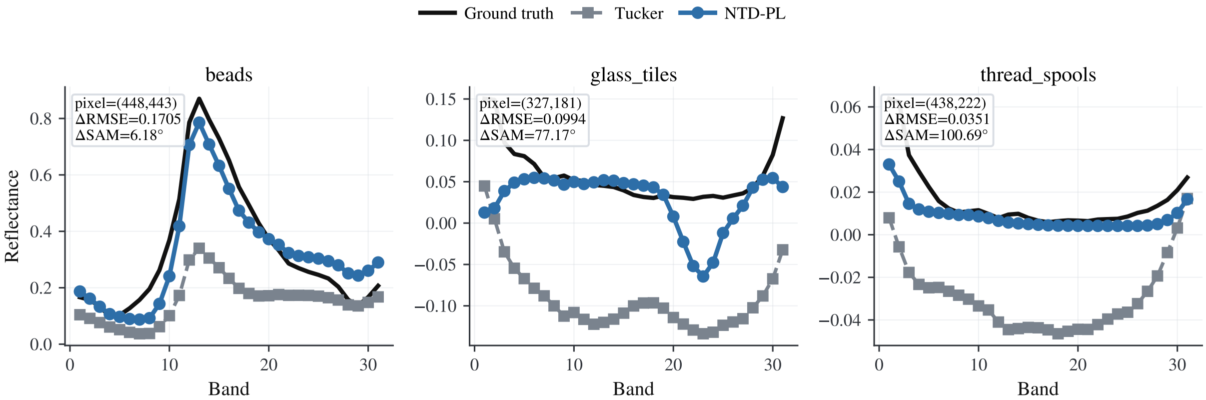


Figure 6: Representative spectral-line comparison for full-observation reconstruction on CAVE. The figure shows the ground-truth and reconstructed spectra for the corresponding pixels. The annotated ΔRMSE and ΔSAM indicate the improvements relative to Tucker.

under all settings, $\rho = 0.1, 0.3, 0.5, 0.7$, NTD-PL consistently outperforms Tucker, with clear improvements in both RMSE* and SAM*. This indicates that the advantage of NTD-PL is not limited to mild correction at low missing rates, but remains stable from mild to severe missingness. In particular, at the highest missing rate, $\rho = 0.7$, the improvement in RMSE* becomes even larger, suggesting that when available observations are sparser, nonlinearity still plays a clear role in recovering missing entries.

Table 5: Main results for random-missing completion on CAVE. Under all four missing rates, NTD-PL outperforms Tucker, showing that the gain directly manifests in the recovery quality at the missing entries.

ρ	Method	RMSE*	NMSE(dB)*	SAM*
0.1	Tucker	0.0263 ± 0.0179	-18.728 ± 4.581	11.82 ± 5.33
	NTD-PL	**0.0229 ± 0.0158**	**-19.909 ± 4.683**	**8.36 ± 3.38**
0.3	Tucker	0.0263 ± 0.0179	-18.707 ± 4.575	14.75 ± 6.33
	NTD-PL	**0.0230 ± 0.0158**	**-19.864 ± 4.664**	**12.47 ± 5.06**
0.5	Tucker	0.0264 ± 0.0179	-18.688 ± 4.582	15.56 ± 6.85
	NTD-PL	**0.0230 ± 0.0158**	**-19.865 ± 4.691**	**13.48 ± 5.75**
0.7	Tucker	0.0266 ± 0.0180	-18.614 ± 4.573	16.02 ± 7.21
	NTD-PL	**0.0219 ± 0.0128**	**-20.043 ± 4.227**	**13.79 ± 5.43**

5.3.4 Mechanism Distinction from Polynomial Post-Processing

The previous two subsections have shown that NTD-PL consistently outperforms Tucker in both full-observation reconstruction and random-missing completion. However, one question still needs to be answered more carefully: where exactly does this gain come from? One possible explanation is that the advantage of NTD-PL may not require joint modeling at all; perhaps one could simply obtain a Tucker reconstruction first and then apply an element-wise polynomial calibration to its output. To distinguish between these possibilities, we introduce a post-processing baseline, *PolyCal*. This method first trains standard Tucker, and then fits a shared element-wise fourth-order polynomial response mapping on its output, thereby post-correcting the entry-level response without changing the low-rank backbone.

Table 6: Scene-level comparison for random-missing completion on CAVE. Under all missing rates, the difference is uniformly defined as Tucker − NTD-PL, so positive values indicate that NTD-PL is better. The results show that the completion gain is significantly consistent at the scene level across different missing rates.

ρ	Metric	Win/Loss/Tie	Median gain	Sign test p	Wilcoxon p
0.1	RMSE*	14/1/0	0.0027	9.77×10^{-4}	1.22×10^{-4}
0.1	SAM*	14/1/0	2.9742	9.77×10^{-4}	8.54×10^{-4}
0.3	RMSE*	14/1/0	0.0027	9.77×10^{-4}	1.22×10^{-4}
0.3	SAM*	13/2/0	1.9194	0.007	0.004
0.5	RMSE*	14/1/0	0.0026	9.77×10^{-4}	1.22×10^{-4}
0.5	SAM*	13/2/0	1.6566	0.007	0.012
0.7	RMSE*	14/1/0	0.0027	9.77×10^{-4}	1.22×10^{-4}
0.7	SAM*	12/3/0	1.3394	0.035	0.048

Table 7: Main table for the mechanism comparison. The three methods are compared on full-observation reconstruction and random-missing completion at $\rho = 0.5$. PolyCal applies a shared element-wise polynomial calibration after Tucker output, whereas NTD-PL jointly models the nonlinear link and the low-rank backbone.

Setting	Method	Param.	RMSE↓	SAM↓
Recon.	Tucker	38,272	0.02606 ± 0.01718	16.4329 ± 7.2207
	PolyCal	38,277	0.02557 ± 0.01661	15.4054 ± 6.3013
	NTD-PL	38,279	**0.02234** ± **0.01497**	**14.4082** ± **6.3010**
Compl.	Tucker	38,272	0.02637 ± 0.01791	15.5569 ± 6.8502
	PolyCal	38,277	0.02587 ± 0.01731	14.3420 ± 5.7617
	NTD-PL	38,277	**0.02299** ± **0.01582**	**13.4841** ± **5.7459**

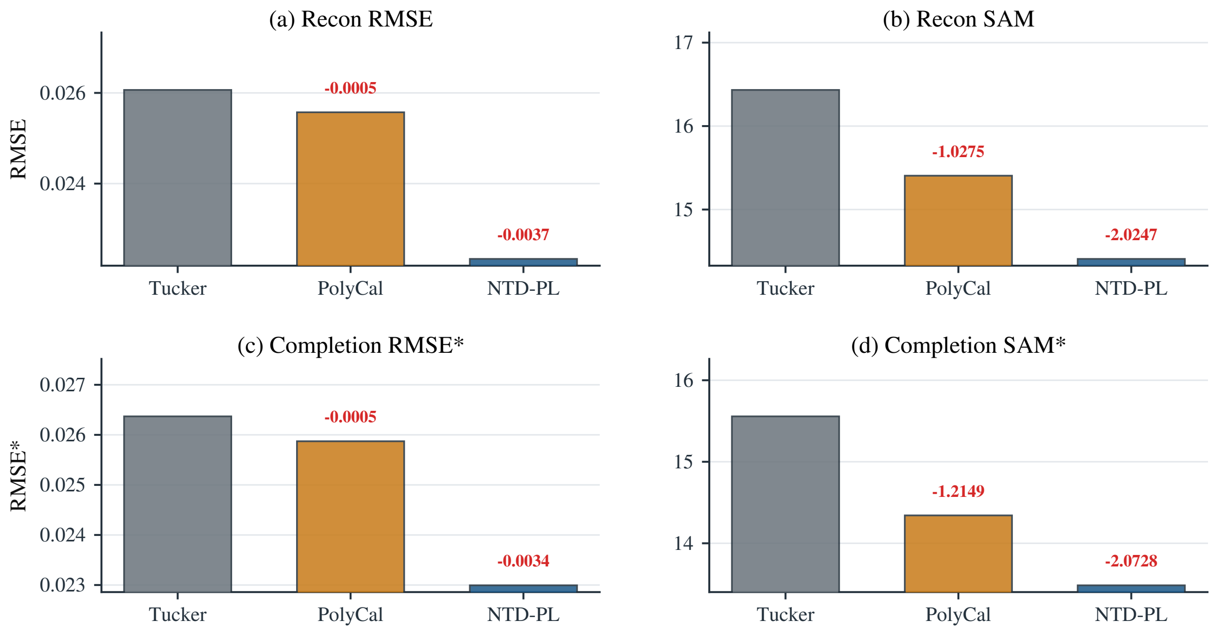


Figure 7: Main figure for mechanism discrimination. The four panels compare RMSE and SAM for full-observation reconstruction, and RMSE* and SAM* for completion at $\rho = 0.5$. All three methods exhibit a consistent improvement relationship, showing that post-processing can only partially approach the gains of NTD-PL and cannot replace its joint modeling.

5.3.5 Where the Advantage Appears

This subsection focuses on a more specific question: where exactly in the image does the additional gain of NTD-PL occur? If its effect were merely a globally smooth correction, the gain should be approximately uniform. If, instead, it corrects the local structures that are hardest for linear low-rank models to fit, then the gain should concentrate near difficult regions and structural boundaries.

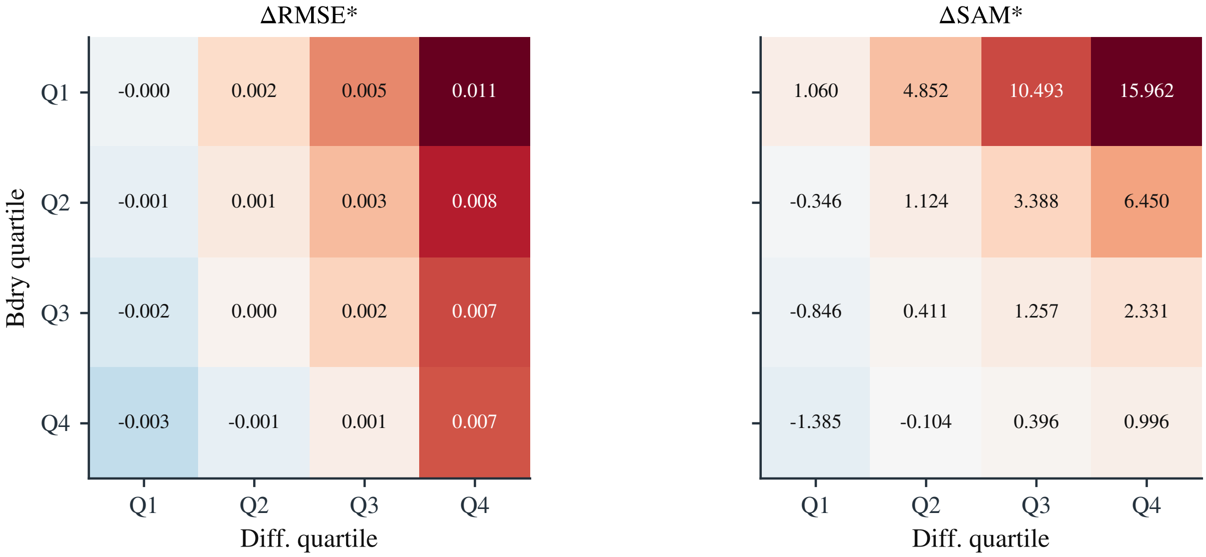


Figure 8: Two-dimensional distribution of the local gain of NTD-PL over Tucker across difficulty quartiles and boundary quartiles. Positive values indicate that NTD-PL is better. One can see that the gain is not uniformly distributed, but is concentrated mainly in the intersection of difficult and boundary-rich regions.

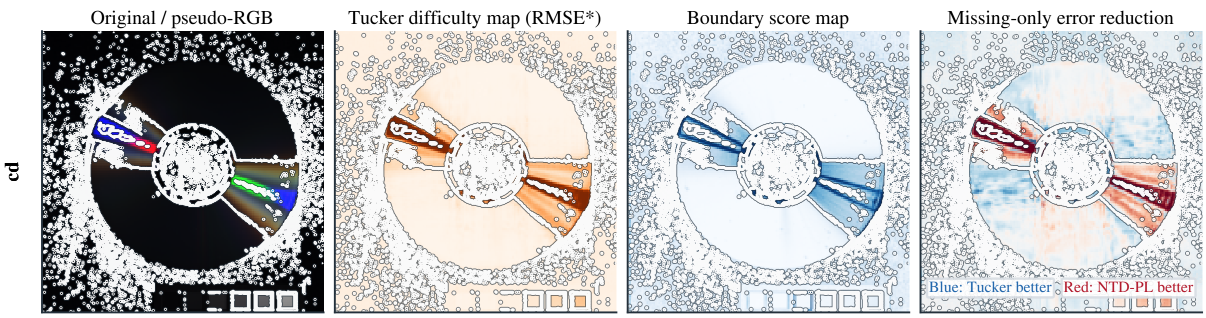


Figure 9: Localization of local gain on the representative `cd` scene. The four columns show, in order, the pseudo-RGB original image, the difficulty map (brighter orange indicates greater difficulty), the boundary map (brighter blue indicates stronger boundaries), and the missing-only error reduction map (red indicates NTD-PL is better and blue indicates Tucker is better). The white contour marks the intersection of the top-difficulty and top-boundary regions. One can see that the positive gain is concentrated mainly at locations where high difficulty and strong boundaries occur simultaneously.

5.3.6 Examination of the Polynomial Degree

To examine which orders provide the main gains in NTD-PL, we scan the maximum polynomial degree P under the same CAVE random-missing completion setting as above, and record both the missing-only metrics and the effective contribution of each degree term. The left panel of Figure 10 shows that completion performance continues to improve as P increases from 1 to 4, but the main gains are concentrated in the low- to medium-order range, and the benefit from further increasing the degree becomes clearly weaker. The right panel further shows that the model contribution is mainly carried by the second- and third-order terms: once $P \geq 3$, these two terms remain dominant, while the fourth-order term contributes only a small marginal correction. This indicates that the main mismatch in real hyperspectral completion does not require very high-order degrees of freedom to describe; explicit nonlinearity of low to medium order is already sufficient to absorb most of the nonlinear residual.

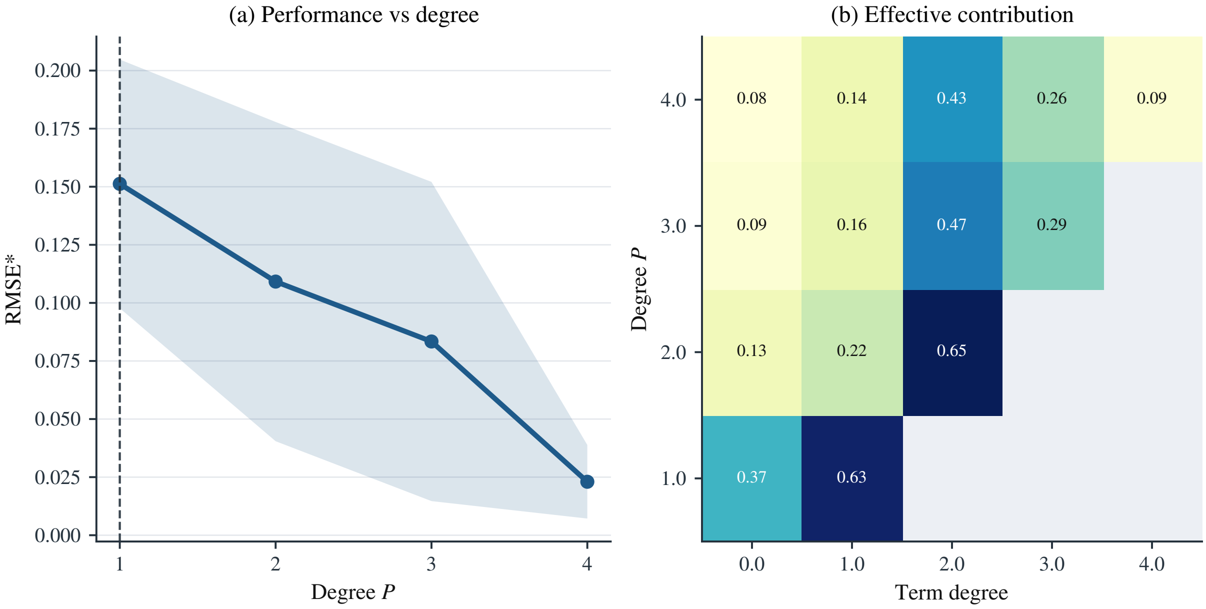


Figure 10: Degree-mechanism analysis for random-missing completion on CAVE. The left panel shows the main missing-entry metrics as a function of the maximum polynomial degree P; the right panel shows the average effective contribution of each degree term under different P.

5.3.7 Cross-Dataset Robustness Validation

To test whether the above conclusions hold only on CAVE, we further conduct external validation on Jasper Ridge, Samson, Urban, and Cuprite, considering both full-observation reconstruction and random-missing completion. All datasets use full-scene settings, global max normalization, and ranks automatically determined by the same compression-ratio matching rule. The completion task uniformly uses a random element-wise missing rate of $\rho = 0.5$. Table 8 shows that the advantage of NTD-PL is not CAVE-specific. On Jasper Ridge, Samson, and Urban, NTD-PL outperforms Tucker in both reconstruction and completion, and ΔNMSE(dB) and ΔSAM mostly improve in the same direction. Overall, under the RMSE metric, 7 out of 8 dataset $\times$ task combinations show positive gains, indicating that the advantage is fairly stable across different scenes and tasks. Cuprite, however, provides a clear boundary case: on this scene, the gains of NTD-PL are substantially weaker, and Tucker is slightly better on some metrics. This indicates that when the data are closer to classical linear mixing and the entry-level nonlinear deviation is weaker, Tucker can already explain the main structure well, and polynomial links can then bring only limited marginal improvement. Therefore, the conclusion about NTD-PL should be understood as follows: it can stably improve Tucker in most real hyperspectral scenes, but this advantage is not unconditional; its effectiveness still depends on the strength of the entry-level nonlinear residual in the data.

Table 8: Main table for cross-dataset robustness validation. In the Recon. rows, Tucker / NTD-PL denotes RMSE; in the Compl. rows, Tucker / NTD-PL denotes RMSE*. Gain(%), ΔNMSE(dB), and ΔSAM are uniformly defined as Tucker − NTD-PL, so positive values indicate that NTD-PL is better.

Dataset	Task	Rank	Tucker	NTD-PL	Gain(%)	ΔNMSE(dB)	ΔSAM(°)
Jasper Ridge	Recon.	(11,11,43)	0.04623	0.04302	**6.93**	**0.623**	**0.981**
	Compl.	(11,11,43)	0.04710	0.04342	**7.82**	**0.708**	**1.156**
Samson	Recon.	(10,10,35)	0.02626	0.02417	**7.95**	**0.719**	**0.057**
	Compl.	(10,10,35)	0.02681	0.02444	**8.86**	**0.806**	**0.310**
Urban	Recon.	(43,43,47)	0.04444	0.04285	**3.59**	**0.318**	**0.288**
	Compl.	(43,43,47)	0.04538	0.04326	**4.67**	**0.415**	**0.488**
Cuprite	Recon.	(35,26,54)	0.01759	0.01770	-0.60	-0.052	-0.037
	Compl.	(35,26,54)	0.01791	0.01781	**0.56**	**0.048**	-0.004

6 Conclusion

This paper was centered on a core question: while preserving the Tucker low-rank backbone as much as possible, how can one introduce effective nonlinearity in a controlled manner? To answer this question, we proposed NTD-PL. The method takes a shared Tucker latent tensor as the structural backbone and imposes an explicit polynomial link at the entry level, thereby compressing nonlinear enhancement into only a few link coefficients. NTD-PL places more emphasis on structural continuity, controllable parameterization, and interpretability of the underlying mechanism.

Around this modeling idea, we provided a complete argument from model to experiments. Theoretically, NTD-PL has a clear nesting relation with standard Tucker: the linear special case degenerates exactly to Tucker, while increasing the degree yields a controlled model expansion. It achieves exact matching under polynomial generation and finite-order approximation with truncation error bounds under analytic generation. Algorithmically, we presented a unified learning framework that can handle both full-observation decomposition and masked completion, and that remains consistent with practical implementation.

The experimental results support these conclusions. In the linear-consistency experiments, restricted NTD-PL almost coincides with Tucker, showing that the model does not fabricate artificial gains in purely linear regimes. In controlled nonlinear residual experiments, the performance trends are consistent with the theoretical expectations. Real hyperspectral experiments further show that, on both full-observation reconstruction and random-missing completion tasks on CAVE, NTD-PL consistently outperforms Tucker under matched parameter budgets, and the advantage is consistent at both the scene-wise and rate-wise levels.

More importantly, the mechanism and spatial evidence explain why the method is effective. The mechanism comparison shows that post-hoc polynomial calibration on Tucker outputs can only partially approximate NTD-PL and cannot replace the joint modeling of nonlinear links with the low-rank backbone. The spatial analysis shows that the gain is not globally uniform smoothing, but is mainly concentrated in difficult local regions with strong boundaries. Degree analysis further shows that the main gains are concentrated in low to medium orders and that marginal improvements gradually saturate as the degree increases, indicating that the additional computational cost of the method is quantifiable and bounded. Cross-dataset results also lead to a clear conclusion: NTD-PL is effective on most real scenes, but not unconditionally so. When the data are closer to linear mixing and the entry-level nonlinear deviation is weaker, its additional gains are correspondingly reduced.

Future work may proceed along three directions. First, one may investigate more stable or more flexible explicit link families, such as orthogonal bases or piecewise bases, to improve numerical conditioning at higher orders. Second, one may develop more efficient mask-aware training and approximate computation to improve large-scale completion efficiency. Third, one may carry out more systematic theoretical analysis on identifiability, generalization error, and adaptive degree selection. Overall, NTD-PL provides an analyzable, implementable, and extensible path for "enhancing Tucker," and offers a reusable starting point for structured nonlinear low-rank modeling.

A Basic Model Properties of NTD-PL

This section corresponds to Section 4.3 of the main text and gathers supplementary proofs for the model-family relations, parameter complexity, and scale indeterminacy of NTD-PL. The main text retains only the statements and their meanings, while the detailed derivations are collected here.

A.1 Reduction to Tucker and Nested Model Families

Proposition 6 (Reduction to Tucker). *If $\beta_1 = 1$ and $\beta_p = 0$ for all $p \neq 1$, then* (5) *reduces to the standard Tucker model, namely*

$$\widehat{\mathcal{Y}} = \mathcal{S} = [\![\mathcal{X}; \mathbf{A}^{(1)}, \dots, \mathbf{A}^{(N)}]\!]. \tag{52}$$

Proof. By the definition of NTD-PL,

$$\widehat{\mathcal{Y}} = \sum_{p=0}^{P} \beta_p \mathcal{S}^{\odot p}, \tag{53}$$

and when $\beta_1 = 1$ and $\beta_p = 0$ for $p \neq 1$, the above expression reduces to

$$\widehat{\mathcal{Y}} = \mathcal{S}. \tag{54}$$

Since $\mathcal{S} = [\![\mathcal{X}; \mathbf{A}^{(1)}, \dots, \mathbf{A}^{(N)}]\!]$, we recover the standard Tucker decomposition form. □

Proposition 7 (Degree Nesting). *For any $P_1 < P_2$, the degree-P_1 NTD-PL model class is a subset of the degree-P_2 model class.*

Proof. Take any degree-P_1 NTD-PL model,

$$\widehat{\mathcal{Y}} = \sum_{p=0}^{P_1} \beta_p \mathcal{S}^{\odot p}. \tag{55}$$

If we view it as a degree-P_2 model, it suffices to set

$$\beta_p = 0, \qquad p = P_1 + 1, \dots, P_2, \tag{56}$$

to obtain exactly the same output. Therefore, every degree-P_1 model belongs to the degree-P_2 model class. □

A.2 Parameter Complexity

Proposition 8 (Parameter Complexity). *Let the Tucker multilinear rank be $(R_1, \dots, R_N)$. Then the total number of parameters in NTD-PL is*

$$\#\text{param} = \prod_{n=1}^{N} R_n + \sum_{n=1}^{N} I_n R_n + (P + 1). \tag{57}$$

Proof. The parameters of NTD-PL consist of three parts:

1. The core tensor $\mathcal{X} \in \mathbb{R}^{R_1 \times \cdots \times R_N}$, whose number of parameters is

$$\prod_{n=1}^{N} R_n; \tag{58}$$

2. The mode factor matrices $\mathbf{A}^{(n)} \in \mathbb{R}^{I_n \times R_n}$, whose total number of parameters is

$$\sum_{n=1}^{N} I_n R_n; \tag{59}$$

3. The polynomial link coefficients $\boldsymbol{\beta} = (\beta_0, \beta_1, \ldots, \beta_P)^\top$, whose number of parameters is

$$P + 1. \tag{60}$$

Adding the three parts together gives

$$\#\text{param} = \prod_{n=1}^{N} R_n + \sum_{n=1}^{N} I_n R_n + (P+1). \tag{61}$$

□

A.3 Scale Indeterminacy and the Normalization Mechanism

We now show that NTD-PL inherits the scale indeterminacy of Tucker decomposition. Therefore, the normalization step in the main text should be understood as a gauge-fixing mechanism rather than as an additional constraint that changes the model class.

Proposition 9 (Scale Invariance)**.** *Let*

$$\mathcal{S} = [\![\mathcal{X}; \mathbf{A}^{(1)}, \ldots, \mathbf{A}^{(N)}]\!]. \tag{62}$$

For a given mode n, take any invertible diagonal matrix $D^{(n)} \in \mathbb{R}^{R_n \times R_n}$ and define

$$\widetilde{\mathbf{A}}^{(n)} = \mathbf{A}^{(n)} D^{(n)}, \qquad \widetilde{\mathcal{X}} = \mathcal{X} \times_n (D^{(n)})^{-1}. \tag{63}$$

Then

$$[\![\widetilde{\mathcal{X}}; \mathbf{A}^{(1)}, \ldots, \mathbf{A}^{(n-1)}, \widetilde{\mathbf{A}}^{(n)}, \mathbf{A}^{(n+1)}, \ldots, \mathbf{A}^{(N)}]\!] = \mathcal{S}. \tag{64}$$

Therefore, for any polynomial coefficients $\boldsymbol{\beta}$, the NTD-PL output

$$\widehat{\mathcal{Y}} = f_{\boldsymbol{\beta}}(\mathcal{S}) \tag{65}$$

remains unchanged.

Proof. Using the associativity of mode products in Tucker reconstruction,

$$(\mathcal{X} \times_n B) \times_n A = \mathcal{X} \times_n (AB),$$

we obtain

$$\begin{aligned}
[\![\widetilde{\mathcal{X}}; \mathbf{A}^{(1)}, \ldots, \widetilde{\mathbf{A}}^{(n)}, \ldots, \mathbf{A}^{(N)}]\!] &= \widetilde{\mathcal{X}} \times_1 \mathbf{A}^{(1)} \times \cdots \times_n \widetilde{\mathbf{A}}^{(n)} \times \cdots \times_N \mathbf{A}^{(N)} && (66)\\
&= \left(\mathcal{X} \times_n (D^{(n)})^{-1}\right) \times_n \left(\mathbf{A}^{(n)} D^{(n)}\right) \times_{m \neq n} \mathbf{A}^{(m)} && (67)\\
&= \mathcal{X} \times_n \left(\left(\mathbf{A}^{(n)} D^{(n)}\right)(D^{(n)})^{-1}\right) \times_{m \neq n} \mathbf{A}^{(m)} && (68)\\
&= \mathcal{X} \times_n \mathbf{A}^{(n)} \times_{m \neq n} \mathbf{A}^{(m)} && (69)\\
&= [\![\mathcal{X}; \mathbf{A}^{(1)}, \ldots, \mathbf{A}^{(N)}]\!] = \mathcal{S}. && (70)
\end{aligned}$$

Hence, collinear rescaling together with core absorption does not change the latent tensor $\mathcal{S}$. Since the output of NTD-PL depends only on $\mathcal{S}$,

$$\widehat{\mathcal{Y}} = f_{\beta}(\mathcal{S}) = \sum_{p=0}^{P} \beta_p \mathcal{S}^{\odot p}, \tag{71}$$

$\widehat{\mathcal{Y}}$ also remains unchanged whenever $\mathcal{S}$ is unchanged. □

Remark 1. *Proposition 9 shows that column normalization of the factor matrices together with absorbing the scale into the core tensor does not change the output of NTD-PL; it only selects a more stable representative from an equivalence class of parameterizations. Since higher-order terms $\mathcal{S}^{\odot p}$ are more sensitive to scale, such normalization is particularly important in polynomial-link models.*

B Expressivity Analysis Under Element-Wise Nonlinear Generation

This section corresponds to Section 4.3.2 of the main text and gives the basic expressivity results of NTD-PL under an element-wise nonlinear generation model. Consistent with the main text, let the true Tucker latent tensor be

$$\mathcal{S}^{\star} = [\![\mathcal{X}^{\star}; \mathbf{A}^{(1)\star}, \ldots, \mathbf{A}^{(N)\star}]\!] \in \mathbb{R}^{I_1 \times \cdots \times I_N}, \tag{72}$$

and let the total number of entries be

$$N_e = \prod_{n=1}^{N} I_n. \tag{73}$$

Assume that the observation tensor is generated by applying some scalar function $g : \mathbb{R} \to \mathbb{R}$ element-wise, that is,

$$\mathcal{Y} = g(\mathcal{S}^{\star}), \qquad \mathcal{Y}_{i_1,\ldots,i_N} = g\Big(\mathcal{S}^{\star}_{i_1,\ldots,i_N}\Big). \tag{74}$$

The NTD-PL model class is written as

$$\widehat{\mathcal{Y}} = f_{\beta}(\mathcal{S}) = \sum_{p=0}^{P} \beta_p \mathcal{S}^{\odot p}, \qquad \mathcal{S} = [\![\mathcal{X}; \mathbf{A}^{(1)}, \ldots, \mathbf{A}^{(N)}]\!], \tag{75}$$

where $\mathcal{S}^{\odot p}$ denotes the element-wise pth power of $\mathcal{S}$, and $\boldsymbol{\beta} = (\beta_0, \ldots, \beta_P)$ are the polynomial link coefficients.

Assumption 1 (Boundedness)**.** *There exists a constant $B > 0$ such that*

$$\|\mathcal{S}^{\star}\|_{\infty} \le B. \tag{76}$$

Assumption 1 is natural in practice, because data preprocessing and scale normalization in the model usually keep the latent Tucker tensor within a bounded range.

B.1 Exact Representation Under Polynomial Generation

Theorem 2 (Exact Representation Under a Polynomial Link)**.** *Suppose the true generating function g is a real-coefficient polynomial of degree at most P, namely*

$$g(x) = \sum_{p=0}^{P} c_p x^p. \tag{77}$$

If the true observation tensor satisfies

$$\mathcal{Y} = g(\mathcal{S}^\star), \qquad \mathcal{S}^\star = [\![\mathcal{X}^\star; \mathbf{A}^{(1)\star}, \ldots, \mathbf{A}^{(N)\star}]\!], \tag{78}$$

then NTD-PL can represent $\mathcal{Y}$ exactly at degree P. More specifically, it suffices to take

$$\mathcal{X} = \mathcal{X}^\star, \qquad \mathbf{A}^{(n)} = \mathbf{A}^{(n)\star} \ \ (n = 1, \ldots, N), \qquad \beta_p = c_p \ \ (p = 0, \ldots, P), \tag{79}$$

in which case

$$\widehat{\mathcal{Y}} = \sum_{p=0}^{P} \beta_p (\mathcal{S}^\star)^{\odot p} = g(\mathcal{S}^\star) = \mathcal{Y}. \tag{80}$$

Proof. For any index $(i_1, \ldots, i_N)$,

$$\widehat{\mathcal{Y}}_{i_1,\ldots,i_N} = \sum_{p=0}^{P} c_p (\mathcal{S}^\star_{i_1,\ldots,i_N})^p = g\Big(\mathcal{S}^\star_{i_1,\ldots,i_N}\Big) = \mathcal{Y}_{i_1,\ldots,i_N}. \tag{81}$$

Since the equality holds for all entries, we have $\widehat{\mathcal{Y}} = \mathcal{Y}$. □

Corollary 1 (Exact Recovery Under Polynomial Nonlinearity)**.** *If the true generation satisfies*

$$\mathcal{Y} = \mathcal{S}^\star + \alpha\, h(\mathcal{S}^\star), \tag{82}$$

where h is a polynomial of degree at most d, then whenever $P \geq d$, there exists a set of NTD-PL parameters such that

$$\widehat{\mathcal{Y}} = \mathcal{Y}. \tag{83}$$

Proof. View

$$g(x) = x + \alpha h(x) \tag{84}$$

as a polynomial of degree at most $\max\{1, d\}$. If $P \geq d$, then the degree of g is at most P, and the conclusion follows directly from Theorem 2. □

B.2 Approximation Bounds Under General Element-Wise Nonlinearity

We now show that when the true nonlinear function is not a polynomial, the approximation error of NTD-PL can be controlled by the approximation error of a one-dimensional scalar function.

Theorem 3 (Tensor Approximation Bound Induced by Scalar Function Approximation)**.** *Assume that Assumption 1 holds, and let*

$$q_P(x) = \sum_{p=0}^{P} \beta_p x^p \tag{85}$$

be any polynomial of degree at most P. If we take the latent variable of NTD-PL to be the true latent tensor, i.e., $\mathcal{S} = \mathcal{S}^\star$, then

$$\|g(\mathcal{S}^\star) - q_P(\mathcal{S}^\star)\|_F \leq \sqrt{N_e} \sup_{|x| \leq B} |g(x) - q_P(x)|. \tag{86}$$

Proof. For any entry $(i_1, \ldots, i_N)$, since $\|\mathcal{S}^\star\|_\infty \leq B$, we have

$$\Big| g\Big(\mathcal{S}^\star_{i_1,\ldots,i_N}\Big) - q_P\Big(\mathcal{S}^\star_{i_1,\ldots,i_N}\Big) \Big| \leq \sup_{|x| \leq B} |g(x) - q_P(x)|. \tag{87}$$

Squaring both sides and summing over all N_e entries yields

$$\|g(\mathcal{S}^\star) - q_P(\mathcal{S}^\star)\|_F^2 \leq N_e \left(\sup_{|x| \leq B} |g(x) - q_P(x)| \right)^2. \tag{88}$$

Taking square roots on both sides gives (86). □

Remark 2. *Theorem 3 shows that, when the true latent variable $\mathcal{S}^\star$ is fixed, the tensor approximation problem of NTD-PL can be reduced to a one-dimensional function approximation problem on the interval $[-B, B]$. Therefore, when the true nonlinearity is an analytic function such as $\sin(\cdot)$ or $\tanh(\cdot)$, the quality of the result is determined by the approximation quality of the corresponding polynomial truncation on that interval.*

B.3 Explicit Truncation Error Bound for Analytic Functions

Theorem 4 (Explicit Truncation Bound for Analytic Link Functions)**.** *Suppose that g is analytic on the complex disk*

$$D(0, \rho) = \{z \in \mathbb{C} : |z| < \rho\} \tag{89}$$

and satisfies

$$0 < B < \rho, \qquad \|\mathcal{S}^\star\|_\infty \le B. \tag{90}$$

Let the Taylor expansion of g at 0 be

$$g(x) = \sum_{p=0}^{\infty} a_p x^p, \tag{91}$$

and let its degree-P truncation be

$$q_P(x) = \sum_{p=0}^{P} a_p x^p. \tag{92}$$

Define

$$M_\rho = \sup_{|z|=\rho} |g(z)|. \tag{93}$$

Then

$$\|g(\mathcal{S}^\star) - q_P(\mathcal{S}^\star)\|_F \le \sqrt{N_e}\, M_\rho \frac{(B/\rho)^{P+1}}{1 - B/\rho}. \tag{94}$$

Proof. By Cauchy's estimate,

$$|a_p| \le \frac{M_\rho}{\rho^p}, \qquad p = 0, 1, 2, \dots \tag{95}$$

Hence for any $|x| \le B$,

$$|g(x) - q_P(x)| = \left| \sum_{p=P+1}^{\infty} a_p x^p \right| \le \sum_{p=P+1}^{\infty} |a_p||x|^p \tag{96}$$

$$\le M_\rho \sum_{p=P+1}^{\infty} (B/\rho)^p = M_\rho \frac{(B/\rho)^{P+1}}{1 - B/\rho}. \tag{97}$$

Applying Theorem 3 then gives

$$\|g(\mathcal{S}^\star) - q_P(\mathcal{S}^\star)\|_F \le \sqrt{N_e}\, M_\rho \frac{(B/\rho)^{P+1}}{1 - B/\rho}. \tag{98}$$

This completes the proof. □

Remark 3. *Theorem 4 provides an explicit error bound. It shows that:*

1. *as P increases, if $B < \rho$, then the truncation error decays geometrically with P;*
2. *for the same degree P, approximation becomes more difficult as the amplitude range B of the latent variable grows;*
3. *this provides a theoretical explanation for the performance differences observed in the experiments under different nonlinear functions, different nonlinear strengths, and different normalization strategies.*

B.4 Relation to the Nonlinear Energy-Ratio Parameter α

In the synthetic data of this paper, the nonlinear-strength parameter α is defined in terms of an *energy ratio*, rather than as a direct coefficient multiplying the nonlinear term. Let the linear base tensor be

$$\mathcal{S}^\star \in \mathbb{R}^{I_1\times\cdots\times I_N}, \tag{99}$$

and let

$$\mathcal{X}^\star = g(\mathcal{S}^\star) \tag{100}$$

denote the tensor obtained after some element-wise nonlinear transformation. To avoid counting the linear component of $\mathcal{X}^\star$ along $\mathcal{S}^\star$ twice as part of the nonlinear component, we first define the orthogonalized residual

$$\mathcal{R}^\star = \mathcal{X}^\star - \frac{\langle \mathcal{X}^\star, \mathcal{S}^\star\rangle}{\|\mathcal{S}^\star\|_F^2}\,\mathcal{S}^\star. \tag{101}$$

If $\|\mathcal{R}^\star\|_F > 0$, we further define the normalized residual

$$\widetilde{\mathcal{R}}^\star = \frac{\|\mathcal{S}^\star\|_F}{\|\mathcal{R}^\star\|_F}\,\mathcal{R}^\star. \tag{102}$$

The observation tensor is then constructed as

$$\mathcal{Y}_\alpha = \sqrt{1-\alpha}\,\mathcal{S}^\star + \sqrt{\alpha}\,\widetilde{\mathcal{R}}^\star, \qquad 0\le\alpha\le 1. \tag{103}$$

This construction is consistent with the data-generation procedure used in the implementation: we first remove from the nonlinear transform its projection onto the linear backbone, then rescale the residual to have the same target energy as the original tensor, and finally mix the two components with coefficients $\sqrt{1-\alpha}$ and $\sqrt{\alpha}$ in terms of energy.

Proposition 10 (Interpretation as an Energy Ratio)**.** *Let $\mathcal{Y}_\alpha$ be defined by* (103)*, and assume that $\|\mathcal{R}^\star\|_F > 0$. Then*

$$\langle \mathcal{S}^\star, \widetilde{\mathcal{R}}^\star\rangle = 0, \qquad \|\widetilde{\mathcal{R}}^\star\|_F = \|\mathcal{S}^\star\|_F, \tag{104}$$

hence

$$\|\mathcal{Y}_\alpha\|_F^2 = \|\mathcal{S}^\star\|_F^2, \tag{105}$$

and the energy proportions of the linear and nonlinear parts satisfy

$$\frac{\|\sqrt{1-\alpha}\,\mathcal{S}^\star\|_F^2}{\|\mathcal{Y}_\alpha\|_F^2} = 1-\alpha, \qquad \frac{\|\sqrt{\alpha}\,\widetilde{\mathcal{R}}^\star\|_F^2}{\|\mathcal{Y}_\alpha\|_F^2} = \alpha. \tag{106}$$

Proof. From (101), we have

$$\langle \mathcal{S}^\star, \mathcal{R}^\star\rangle = 0, \tag{107}$$

hence

$$\langle \mathcal{S}^\star, \widetilde{\mathcal{R}}^\star\rangle = 0. \tag{108}$$

From (102), we also have

$$\|\widetilde{\mathcal{R}}^\star\|_F = \|\mathcal{S}^\star\|_F. \tag{109}$$

Therefore,

$$\begin{aligned}
\|\mathcal{Y}_\alpha\|_F^2 &= \left\|\sqrt{1-\alpha}\,\mathcal{S}^\star + \sqrt{\alpha}\,\widetilde{\mathcal{R}}^\star\right\|_F^2 && (110)\\
&= (1-\alpha)\|\mathcal{S}^\star\|_F^2 + \alpha\|\widetilde{\mathcal{R}}^\star\|_F^2 + 2\sqrt{\alpha(1-\alpha)}\langle \mathcal{S}^\star, \widetilde{\mathcal{R}}^\star\rangle && (111)\\
&= (1-\alpha)\|\mathcal{S}^\star\|_F^2 + \alpha\|\mathcal{S}^\star\|_F^2 && (112)\\
&= \|\mathcal{S}^\star\|_F^2, && (113)
\end{aligned}$$

which gives (105). Computing the proportion of the total energy carried by each part then yields (106). □

We next analyze how the truncation error of NTD-PL varies with α under this energy-ratio definition.

Corollary 2 (Relation Between Truncation Error and α)**.** *Suppose*

$$\mathcal{X}^\star = g(\mathcal{S}^\star), \qquad \|\mathcal{S}^\star\|_\infty \le B, \tag{114}$$

and define

$$c_g = \frac{\langle \mathcal{X}^\star, \mathcal{S}^\star \rangle}{\|\mathcal{S}^\star\|_F^2}, \qquad \eta_g = \frac{\|\mathcal{S}^\star\|_F}{\|\mathcal{R}^\star\|_F}, \tag{115}$$

where $\mathcal{R}^\star$ is defined by (101)*. Let q_P be a polynomial approximation to g on the interval $[-B, B]$ of degree at most P, satisfying*

$$\varepsilon_P(B) := \sup_{|s|\le B} |g(s) - q_P(s)|, \tag{116}$$

and define

$$a_\alpha = \sqrt{1-\alpha} - \sqrt{\alpha}\,\eta_g c_g, \qquad b_\alpha = \sqrt{\alpha}\,\eta_g. \tag{117}$$

Then there exists a corresponding degree-P representation

$$\widehat{\mathcal{Y}}_{\alpha,P} = a_\alpha \mathcal{S}^\star + b_\alpha q_P(\mathcal{S}^\star), \tag{118}$$

such that

$$\|\mathcal{Y}_\alpha - \widehat{\mathcal{Y}}_{\alpha,P}\|_F \le \sqrt{\alpha}\,\eta_g\,\sqrt{N_e}\,\varepsilon_P(B), \tag{119}$$

where

$$N_e = \prod_{n=1}^{N} I_n. \tag{120}$$

Furthermore,

$$\|\mathcal{Y}_\alpha - \widehat{\mathcal{Y}}_{\alpha,P}\|_F^2 \le \alpha\,\eta_g^2\,N_e\,\varepsilon_P(B)^2. \tag{121}$$

Proof. From (101)–(102),

$$\widetilde{\mathcal{R}}^\star = \eta_g(\mathcal{X}^\star - c_g\mathcal{S}^\star) = \eta_g(g(\mathcal{S}^\star) - c_g\mathcal{S}^\star). \tag{122}$$

Substituting this into (103) yields

$$\begin{aligned}
\mathcal{Y}_\alpha &= \sqrt{1-\alpha}\,\mathcal{S}^\star + \sqrt{\alpha}\,\eta_g(g(\mathcal{S}^\star) - c_g\mathcal{S}^\star) && (123)\\
&= (\sqrt{1-\alpha} - \sqrt{\alpha}\,\eta_g c_g)\mathcal{S}^\star + \sqrt{\alpha}\,\eta_g\,g(\mathcal{S}^\star) && (124)\\
&= a_\alpha\mathcal{S}^\star + b_\alpha g(\mathcal{S}^\star). && (125)
\end{aligned}$$

Hence

$$\mathcal{Y}_\alpha - \widehat{\mathcal{Y}}_{\alpha,P} = b_\alpha(g(\mathcal{S}^\star) - q_P(\mathcal{S}^\star)). \tag{126}$$

Using $|b_\alpha| = \sqrt{\alpha}\,\eta_g$ together with

$$\|g(\mathcal{S}^\star) - q_P(\mathcal{S}^\star)\|_F \le \sqrt{N_e}\,\varepsilon_P(B), \tag{127}$$

we obtain

$$\|\mathcal{Y}_\alpha - \widehat{\mathcal{Y}}_{\alpha,P}\|_F \le \sqrt{\alpha}\,\eta_g\,\sqrt{N_e}\,\varepsilon_P(B). \tag{128}$$

Squaring both sides gives (121). □

Remark 4. *Corollary 2 shows that under the energy-ratio generation mechanism adopted in this paper:*

1. the upper bound of the truncation error in Frobenius norm scales with $\sqrt{\alpha}$;

2. the upper bounds of squared error and NMSE-type metrics scale with α;

3. if g itself is a polynomial of degree at most P, then $\varepsilon_P(B) = 0$, and exact representation holds for any $\alpha \in [0, 1]$;

4. if g is analytic, then the truncation error is affected not only by α, but also by the polynomial degree P, the input amplitude range B, and the radius of analyticity.

Moreover, the coefficient

$$\eta_g = \frac{\|\mathcal{S}^\star\|_F}{\|\mathcal{R}^\star\|_F} \tag{129}$$

also reflects the effective strength of the orthogonal nonlinear residual in the true nonlinearity: when the nonlinear residual is weak, the residual direction must be amplified more to achieve the same energy ratio α, and finite-order polynomial approximation becomes correspondingly more difficult.

Remark 5. *When $\|\mathcal{R}^\star\|_F = 0$, the selected nonlinear transformation produces no effective nonlinear residual on the current input. In this case the generation process degenerates to a purely linear one, and α no longer changes the directional structure of the observation tensor.*

C Optimization Supplement

C.1 Uniqueness of the $\boldsymbol{\beta}$ Subproblem

This subsection corresponds to Section 4.4 of the main text and proves that, when $\lambda_\beta > 0$, the polynomial-link coefficient subproblem for $\boldsymbol{\beta}$ in NTD-PL has a unique solution.

Fix the latent tensor

$$\mathcal{S} = [\![\mathcal{X}; \mathbf{A}^{(1)}, \dots, \mathbf{A}^{(N)}]\!], \tag{130}$$

the current polynomial degree P, and the observation set Ω. Consider the optimization problem

$$\min_{\boldsymbol{\beta}\in\mathbb{R}^{P+1}} \left\| \Omega \odot \left(\mathcal{Y} - \sum_{p=0}^{P} \beta_p \mathcal{S}^{\odot p} \right) \right\|_F^2 + \lambda_\beta \|\boldsymbol{\beta}\|_2^2, \qquad \lambda_\beta > 0. \tag{131}$$

Let $M_\Omega = |\Omega|$. After flattening the observed entries, we obtain the observation vector

$$\mathbf{y}_\Omega \in \mathbb{R}^{M_\Omega}, \tag{132}$$

and the corresponding latent-tensor values

$$\mathbf{s}_\Omega = (s_1, \dots, s_{M_\Omega})^\top \in \mathbb{R}^{M_\Omega}. \tag{133}$$

Based on this, we construct the design matrix

$$\Phi_\Omega = \begin{bmatrix} 1 & s_1 & s_1^2 & \cdots & s_1^P \\ 1 & s_2 & s_2^2 & \cdots & s_2^P \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & s_{M_\Omega} & s_{M_\Omega}^2 & \cdots & s_{M_\Omega}^P \end{bmatrix} \in \mathbb{R}^{M_\Omega \times (P+1)}. \tag{134}$$

Then (131) can be rewritten as the standard ridge-regression problem

$$\min_{\boldsymbol{\beta}\in\mathbb{R}^{P+1}} \|\mathbf{y}_\Omega - \Phi_\Omega \boldsymbol{\beta}\|_2^2 + \lambda_\beta \|\boldsymbol{\beta}\|_2^2. \tag{135}$$

Proposition 11 (Unique Solution of the Regularized $\boldsymbol{\beta}$ Subproblem)**.** *When $\lambda_\beta > 0$, problem* (135) *has a unique minimizer, given by*

$$\boldsymbol{\beta}^\star = \left(\Phi_\Omega^\top \Phi_\Omega + \lambda_\beta I\right)^{-1} \Phi_\Omega^\top \mathbf{y}_\Omega. \tag{136}$$

Proof. Define the objective function

$$f(\boldsymbol{\beta}) = \|\mathbf{y}_\Omega - \Phi_\Omega \boldsymbol{\beta}\|_2^2 + \lambda_\beta \|\boldsymbol{\beta}\|_2^2. \tag{137}$$

Expanding it gives

$$f(\boldsymbol{\beta}) = \mathbf{y}_\Omega^\top \mathbf{y}_\Omega - 2\mathbf{y}_\Omega^\top \Phi_\Omega \boldsymbol{\beta} + \boldsymbol{\beta}^\top \Phi_\Omega^\top \Phi_\Omega \boldsymbol{\beta} + \lambda_\beta \boldsymbol{\beta}^\top \boldsymbol{\beta}. \tag{138}$$

Therefore its gradient is

$$\nabla f(\boldsymbol{\beta}) = 2\left(\Phi_\Omega^\top \Phi_\Omega + \lambda_\beta I\right)\boldsymbol{\beta} - 2\Phi_\Omega^\top \mathbf{y}_\Omega, \tag{139}$$

and its Hessian is

$$\nabla^2 f(\boldsymbol{\beta}) = 2\left(\Phi_\Omega^\top \Phi_\Omega + \lambda_\beta I\right). \tag{140}$$

Since $\Phi_\Omega^\top \Phi_\Omega$ is positive semidefinite and $\lambda_\beta I$ is positive definite whenever $\lambda_\beta > 0$, we have

$$\Phi_\Omega^\top \Phi_\Omega + \lambda_\beta I \succ 0. \tag{141}$$

Hence $f(\boldsymbol{\beta})$ is strictly convex and therefore has a unique minimizer. Setting the gradient to zero yields the normal equation

$$\left(\Phi_\Omega^\top \Phi_\Omega + \lambda_\beta I\right)\boldsymbol{\beta} = \Phi_\Omega^\top \mathbf{y}_\Omega. \tag{142}$$

Since the coefficient matrix is positive definite and invertible, the unique solution is

$$\boldsymbol{\beta}^\star = \left(\Phi_\Omega^\top \Phi_\Omega + \lambda_\beta I\right)^{-1} \Phi_\Omega^\top \mathbf{y}_\Omega. \tag{143}$$

This completes the proof. □

Remark 6. *Proposition 11 shows that, when the latent tensor $\mathcal{S}$ is fixed, the update of $\boldsymbol{\beta}$ is a low-dimensional, strongly convex subproblem with a closed-form solution. In particular, uniqueness no longer depends on whether the observations are sufficiently rich, nor does it require the design matrix Φ_Ω to have full column rank; the regularization term $\lambda_\beta I$ directly guarantees invertibility of the system matrix. This is also why regularization is especially important under missing observations and high-order polynomial settings.*

D Supplementary Experimental Figures

This section supplements the figures used in Chapter 5 of the main text to help illustrate the experimental phenomena.

D.1 Supplementary Synthetic Experiments

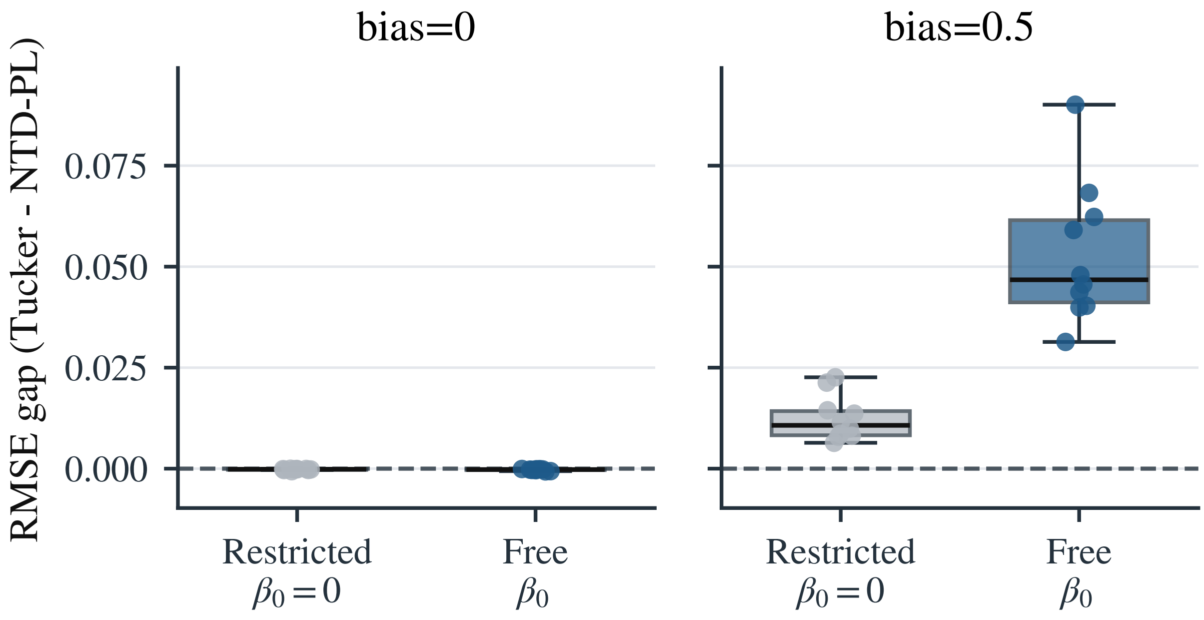


Figure 11: Distribution of paired differences in the linear-consistency experiment. The figure compares the paired error gap between Tucker and NTD-PL across random seeds under different settings, supplementing the scene-wise mean conclusion in Table 1. One can see that when `bias=0`, the paired differences between Tucker and NTD-PL without a constant term are concentrated near zero; when an affine bias is present, a more obvious systematic gap appears between the model with restricted β_0 and Tucker.

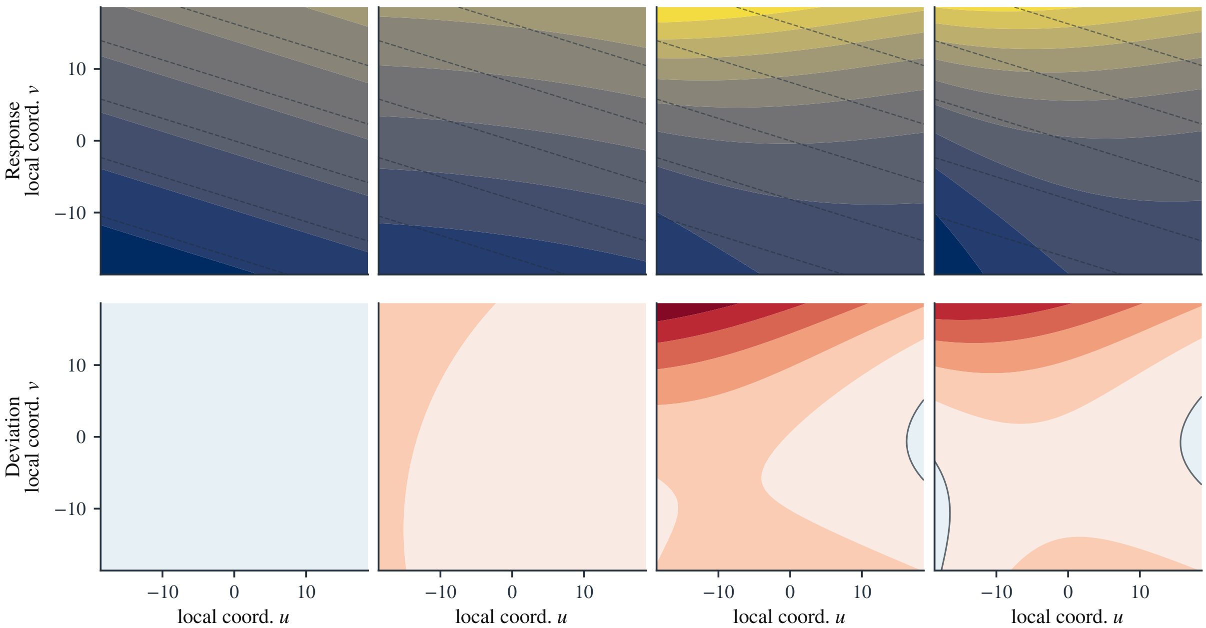


Figure 12: Learned local response maps and deviation maps relative to $P = 1$ under different maximum polynomial degrees P, for a fixed two-dimensional Tucker local patch. The top row shows the local responses, and the bottom row shows the deviations relative to the linear baseline.

D.2 Supplementary Real Hyperspectral Experiments

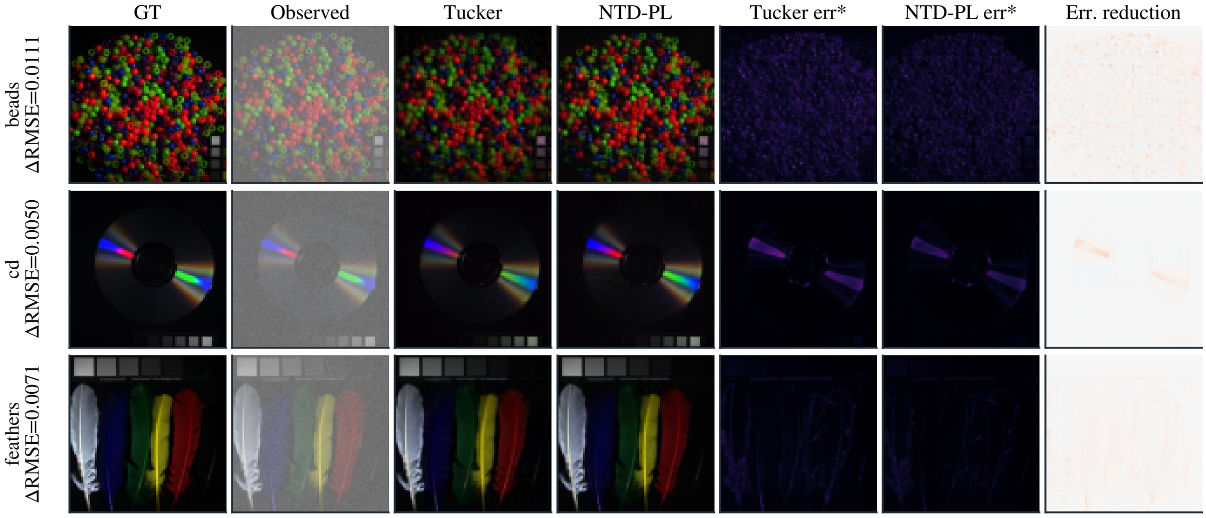


Figure 13: Representative spatial visualization for random-missing completion on CAVE. The columns show, in order, the original image, the masked observation input, Tucker completion, NTD-PL completion, the missing-only error of Tucker, the missing-only error of NTD-PL, and the missing-only error-reduction map. The last column is defined as Tucker − NTD-PL, so positive values indicate that NTD-PL further reduces the completion error at the missing entries.

References

[1] Dawon Ahn, Uday Singh Saini, Evangelos E Papalexakis, and Ali Payani. Neural additive tensor decomposition for sparse tensors. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, pages 14–23, 2024.

[2] Andrzej Cichocki, Danilo P. Mandic, Anh-Huy Phan, Cesar F. Caiafa, Guoxu Zhou, Qibin Zhao, and Lieven De Lathauwer. Tensor decompositions for signal processing applications: From two-way to multiway component analysis. *IEEE Signal Processing Magazine*, 32(2): 145–163, 2015. doi: 10.1109/MSP.2013.2297439.

[3] Columbia CAVE Laboratory. Multispectral image database. `https://cave.cs.columbia.edu/repository/Multispectral`. Accessed: 2026-04-13.

[4] Curtiss O. Davis, Maria Kavanaugh, Ricardo Letelier, W. Paul Bissett, and David Kohler. Spatial and spectral resolution considerations for imaging coastal waters. In *Coastal Ocean Remote Sensing*, volume 6680 of *Proceedings of SPIE*, page 66800P, 2007. doi: 10.1117/12.734288.

[5] Lieven De Lathauwer, Bart De Moor, and Joos Vandewalle. A multilinear singular value decomposition. *SIAM Journal on Matrix Analysis and Applications*, 21(4):1253–1278, 2000. doi: 10.1137/S0895479896305696.

[6] Lieven De Lathauwer, Bart De Moor, and Joos Vandewalle. A multilinear singular value decomposition. *SIAM journal on Matrix Analysis and Applications*, 21(4):1253–1278, 2000.

[7] Jian Deng, Haotian Liu, Kim Batselier, Yu-Kwong Kwok, and Ngai Wong. Storm: a nonlinear model order reduction method via symmetric tensor decomposition. In *2016 21st Asia and South Pacific design automation conference (ASP-DAC)*, pages 557–562. IEEE, 2016.

[8] Dmytro Derkach, Adria Ruiz, and Federico M Sukno. Tensor decomposition and non-linear manifold modeling for 3d head pose estimation. *International Journal of Computer Vision*, 127(10):1565–1585, 2019.

[9] Geospatial Research Laboratory (U.S.). Hypercube sample data set: Hydice sensor imagery urban. `https://hdl.handle.net/11681/2925`, 2015. Dataset.

[10] Arthur E Hoerl and Robert W Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970.

[11] David Hong, Tamara G Kolda, and Jed A Duersch. Generalized canonical polyadic tensor decomposition. *SIAM review*, 62(1):133–163, 2020.

[12] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[13] Tamara G. Kolda and Brett W. Bader. Tensor decompositions and applications. *SIAM Review*, 51(3):455–500, 2009. doi: 10.1137/07070111X.

[14] Bin Liu, Lirong He, Shandian Zhe, Yingmin Li, and Zenglin Xu. Deepcp: Flexible nonlinear tensor decomposition. In *NeurIPS Workshops*, volume 3, 2017.

[15] Haotian Liu, Luca Daniel, and Ngai Wong. Model reduction and simulation of nonlinear circuits via tensor decomposition. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 34(7):1059–1069, 2015.

[16] Ji Liu, Przemyslaw Musialski, Peter Wonka, and Jieping Ye. Tensor completion for estimating missing values in visual data. *IEEE transactions on pattern analysis and machine intelligence*, 35(1):208–220, 2012.

[17] Ji Liu, Przemyslaw Musialski, Peter Wonka, and Jieping Ye. Tensor completion for estimating missing values in visual data. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 35(1):208–220, 2013. doi: 10.1109/TPAMI.2012.39.

[18] Ivan V. Oseledets. Tensor-train decomposition. *SIAM Journal on Scientific Computing*, 33 (5):2295–2317, 2011.

[19] Zhimeng Pan, Zheng Wang, and Shandian Zhe. Streaming nonlinear bayesian tensor decomposition. In *Conference on Uncertainty in Artificial Intelligence*, pages 490–499. PMLR, 2020.

[20] Ronald G. Resmini, Mary E. Kappus, W. S. Aldrich, John C. Harsanyi, and M. Anderson. Mineral mapping with hyperspectral digital imagery collection experiment (hydice) sensor data at cuprite, nevada, u.s.a. *International Journal of Remote Sensing*, 18(7):1553–1570, 1997. doi: 10.1080/014311697218278.

[21] D. A. Roberts, M. O. Smith, and J. B. Adams. Green vegetation, nonphotosynthetic vegetation, and soils in aviris data. *Remote Sensing of Environment*, 44(2–3):255–269, 1993. doi: 10.1016/0034-4257(93)90020-X.

[22] Vishwanath Saragadam, Randall Balestriero, Ashok Veeraraghavan, and Richard G Baraniuk. Deeptensor: Low-rank tensor decomposition with deep network priors. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(12):10337–10348, 2024.

[23] Aaron Schein, Mingyuan Zhou, David Blei, and Hanna Wallach. Bayesian poisson tucker decomposition for learning the structure of international relations. In *International Conference on Machine Learning*, pages 2810–2819. PMLR, 2016.

[24] Zhi-Wei Shi, Yu-Bang Zheng, Yi Zhang, and Heng-Chao Li. Multidimensional nonlinear transform-based tensor representation for high-dimensional image reconstruction. *Pattern Recognition*, 168:111734, 2025.

[25] Nicholas D. Sidiropoulos, Lieven De Lathauwer, Xiao Fu, Kejun Huang, Evangelos E. Papalexakis, and Christos Faloutsos. Tensor decomposition for signal processing and machine learning. *IEEE Transactions on Signal Processing*, 65(13):3551–3582, 2017. doi: 10.1109/TSP.2017.2690524.

[26] Ledyard R Tucker. Some mathematical notes on three-mode factor analysis. *Psychometrika*, 31(3):279–311, 1966.

[27] Uras Varolgunes, Dan Zhou, Dantong Yu, and Ajim Uddin. Nmtucker: Non-linear matryoshka tucker decomposition for financial time series imputation. In *Proceedings of the Fourth ACM International Conference on AI in Finance*, pages 516–523, 2023.

[28] Dingheng Wang, Guangshe Zhao, Hengnu Chen, Zhexian Liu, Lei Deng, and Guoqi Li. Nonlinear tensor train format for deep neural network compression. *Neural Networks*, 144: 320–333, 2021.

[29] Xian Wu, Baoxu Shi, Yuxiao Dong, Chao Huang, and Nitesh V Chawla. Neural tensor factorization for temporal interaction learning. In *Proceedings of the Twelfth ACM international conference on web search and data mining*, pages 537–545, 2019.

[30] Shuang Xu, Jiangjun Peng, Teng-Yu Ji, Xiangyong Cao, Kai Sun, Rongrong Fei, and Deyu Meng. Stacked tucker decomposition with multi-nonlinear products for remote sensing imagery inpainting. *IEEE Transactions on Geoscience and Remote Sensing*, 62:1–13, 2024.

[31] Zenglin Xu, Feng Yan, et al. Infinite tucker decomposition: Nonparametric bayesian models for multiway data analysis. *arXiv preprint arXiv:1108.6296*, 2011.

[32] Linxiao Yang, Jun Fang, Hongbin Li, and Bing Zeng. An iterative reweighted method for tucker decomposition of incomplete tensors. *IEEE Transactions on Signal Processing*, 64 (18):4817–4829, 2016.

[33] Fumihito Yasuma, Tomoo Mitsunaga, Daisuke Iso, and Shree K. Nayar. Generalized assorted pixel camera: Postcapture control of resolution, dynamic range, and spectrum. *IEEE Transactions on Image Processing*, 19(9):2241–2253, 2010. doi: 10.1109/TIP.2010.2046811.

[34] Zheng Zhai, Hengchao Chen, and Qiang Sun. Quadratic matrix factorization with applications to manifold learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(9):6384–6401, 2024.

[35] Qibin Zhao, Guoxu Zhou, Shengli Xie, Liqing Zhang, and Andrzej Cichocki. Tensor ring decomposition. *arXiv preprint arXiv:1606.05535*, 2016.

[36] Shandian Zhe, Kai Zhang, Pengyuan Wang, Kuang-chih Lee, Zenglin Xu, Yuan Qi, and Zoubin Ghahramani. Distributed flexible nonlinear tensor factorization. *Advances in neural information processing systems*, 29, 2016.